

01 개발자매뉴얼

기준 Repository: https://github.com/freeangelsun/202412_01_CPF | Branch: master | Commit: 77db10ad9aff44ee422795080fb2e96b364c9d65 (08_01) | 기준일: 2026-08-07 Asia/Seoul

문서 상태: 정적 문서·Source 정합성 검토 대상. 실제 Runtime/DB/Browser 에서 실행하지 않은 항목은 본문에서 미검증으로 구분한다.

빠른 탐색

예제·실습 1.2 예제 업무 7.2 PostgreSQL 예제 9.2 Application Service 예제 16. Attachment 예제 20. Fault Injection 실습 실습 A — 중복 요청 실습 B — Version 충돌 실습 C — Consumer Kill 실습 D — HTTP 응답 유실 부록 A. PAY 예제 전체 연결 Source 부록 B. 온라인·공통·외부 연계 EDU 45 개 EDU-ONL-01 — Generator Dry Run 과 충돌 검토 EDU-ONL-02 — secure-api Profile 조립 EDU-ONL-03 — 조건 검색과 Paging EDU-ONL-04 — 상세 조회와 Masking EDU-ONL-05 — Versioned 상태 변경 EDU-ONL-06 — Idempotency 동일 요청 재호출 EDU-ONL-07 — Idempotency Key Payload 충돌 EDU-ONL-08 — Optimistic Lock 동시 수정 EDU-ONL-09 — 업무 Transaction 과 Audit 외 37 개 - 아래 전체 목차 사용	오류·복구 10. 오류 응답 계약 부록 C. 오류·상태·HTTP 판정표 F.6 파일 100 만 행 중 23 행이 오류다	Source·API·설정 5. PAY Source 구조 7. DB Schema 와 Migration 7.2 PostgreSQL 예제 8. Query API 9. Command API 9.3 API 응답 14. OpenAPI WebMVC 부록 A. PAY 예제 전체 연결 Source A.4 JDBC Optimistic Update EDU-ONL-02 — secure-api Profile 조립 EDU-ONL-44 — OpenAPI Snapshot Refresh E.7 Query API 와 검색 방어 E.8 DB Table 계약 E.12 DB·Log·Trace·Audit 확인 F.5 Consumer 가 DB Commit 뒤 종료됐다 F.9 DB Migration 뒤 구 버전 Instance 가 남았다 F.15 OpenAPI Operation ID 가 중복됐다
--	--	--

전체 문단 목차

CPF 개발자 매뉴얼	EDU-ONL-10 — Outbox 원자 저장
1. 학습 목표와 준비	EDU-ONL-11 — Outbox Publisher Lease
1.1 선수 지식	EDU-ONL-12 — Inbox 중복 소비 차단
1.2 예제 업무	EDU-ONL-13 — Broker DLQ 와 Replay
2. 개발 환경과 기준 확인	EDU-ONL-14 — Kafka Consumer Process Kill
3. Profile 과 Capability 선택	EDU-ONL-15 — RabbitMQ Provider 교체
4. Generator Dry Run	EDU-ONL-16 — JMS Provider 교체
5. PAY Source 구조	EDU-ONL-17 — IBM MQ 연계
5.1 계층별 책임	EDU-ONL-18 — HTTP Timeout Budget
6. Domain 상태 모델	EDU-ONL-19 — HTTP 응답 유실
7. DB Schema 와 Migration	EDU-ONL-20 — Circuit Breaker 전이
7.1 논리 Table	EDU-ONL-21 — TCP Length Framing
7.2 PostgreSQL 예제	EDU-ONL-22 — TCP CRLF Framing
7.3 Optimistic Update	EDU-ONL-23 — TCP STX/ETX Framing
8. Query API	EDU-ONL-24 — Fixed-length 전문
8.1 DTO	EDU-ONL-25 — ISO8583 전문
8.2 Controller	EDU-ONL-26 — Attachment Upload
8.3 Negative Test	EDU-ONL-27 — Archive 안전 해제
9. Command API	EDU-ONL-28 — Tabular Preview·Apply
9.1 입력	EDU-ONL-29 — SFTP 수신
9.2 Application Service 예제	EDU-ONL-30 — Email Notification
9.3 API 응답	EDU-ONL-31 — SMS SPI
10. 오류 응답 계약	EDU-ONL-32 — Caffeine Local Cache
11. Outbox·Kafka·Inbox	EDU-ONL-33 — Valkey Distributed Cache
11.1 Event	EDU-ONL-34 — Durable Cache Invalidation
11.2 Producer 흐름	EDU-ONL-35 — Feature Flag Override
11.3 Consumer 흐름	EDU-ONL-36 — Secret Rotation
11.4 Fault Test	EDU-ONL-37 — Service Identity
12. 외부기관 HTTP 와 결과 불명	EDU-ONL-38 — Browser Session·CSRF
12.1 Port	EDU-ONL-39 — Data Scope 와 Masking
12.2 Attempt 저장	EDU-ONL-40 — Business Calendar
12.3 상태 분류	EDU-ONL-41 — Time·Deadline
12.4 WireMock Fault 예	EDU-ONL-42 — Data Quality Quarantine
13. Local·Remote 동일 계약	EDU-ONL-43 — 정정 승인 Snapshot
14. OpenAPI WebMVC	EDU-ONL-44 — OpenAPI Snapshot Refresh
15. Security·Data Scope·Masking	EDU-ONL-45 — Local·Remote 계약 동등성
15.1 사용자와 서비스 Identity	부록 C. 오류·상태·HTTP 판정표

15.2 Permission 판정	부록 D. 코드 리뷰에서 반드시 찾을 결함
15.3 Field 암호화	부록 E. PAY 교육 프로젝트를 실제 업무로 바꾸는 전체 절차
16. Attachment 예제	E.1 학습 결과와 완료 판정
16.1 처리 순서	E.2 고객 프로젝트 Directory
16.2 상태	E.3 Gradle 조립 예시
17. 데이터 품질 격리·정정·Replay	E.4 Domain 상태 모델
18. 시간·Deadline	E.5 Port 경계
19. Test 전략	E.6 Application Service 의 분기
19.1 실행 예	E.7 Query API 와 검색 방어
20. Fault Injection 실습	E.8 DB Table 계약
실습 A — 중복 요청	E.9 MyBatis Update Count 판정
실습 B — Version 충돌	E.10 외부기관 Attempt 상태기계
실습 C — Consumer Kill	E.11 PowerShell 요청과 예상 응답
실습 D — HTTP 응답 유실	같은 Key·같은 Body: 같은 methodId와 operationId 계열 결과, 부작용 1 회
21. ADM 확인	E.12 DB·Log·Trace·Audit 확인
22. 운영 인계표	E.13 자동 Test 전체 목록
23. 고객 업무로 바꾸는 부분	E.14 Fault Injection 실행 순서
24. 개발자 자체 검수	E.15 고객 업무 전환표
25. 08_01 R6 계약 반영: 승인 정정과 먹등성	부록 F. 개발 중 자주 만나는 20 개 판단 사례
부록 A. PAY 예제 전체 연결 Source	F.1 검색 결과가 2 만 건을 넘는다
A.1 요청·응답 DTO	F.2 두 사용자가 같은 신청을 승인한다
A.2 Command Controller	F.3 기관이 처리했지만 HTTP 응답이 끊겼다
A.3 Application Service 의 처리 순서	F.4 Broker 는 10 분간 사용할 수 없다
A.4 JDBC Optimistic Update	F.5 Consumer 가 DB Commit 뒤 종료됐다
A.5 Outbox Publisher	F.6 파일 100 만 행 중 23 행이 오류다
A.6 Inbox Consumer	F.7 권한이 회수됐지만 Session 이 남았다
A.7 외부기관 Attempt 와 결과 불명	F.8 Secret Key Version 을 교체한다
A.8 Error Response	F.9 DB Migration 뒤 구 버전 Instance 가 남았다
A.9 통합 Test 시나리오	F.10 같은 업무를 Local 에서 Remote 로 분리한다
부록 B. 온라인·공통·외부 연계 EDU 45 개	F.11 다중 Instance 에서 Cache 값이 다르다
EDU-ONL-01 — Generator Dry Run 과 충돌 검토	F.12 외부 요청 Body 가 50MB 다
EDU-ONL-02 — secure-api Profile 조립	F.13 고객이 같은 파일을 다시 업로드했다
EDU-ONL-03 — 조건 검색과 Paging	F.14 정정 승인 후 원본 Row 가 바뀌었다
EDU-ONL-04 — 상세 조회와 Masking	F.15 OpenAPI Operation ID 가 중복됐다
EDU-ONL-05 — Versioned 상태 변경	F.16 시간대가 다른 기관과 영업일을 계산한다
EDU-ONL-06 — Idempotency 동일 요청 재호출	F.17 개인정보 Export 가 오래 걸린다
EDU-ONL-07 — Idempotency Key Payload 충돌	F.18 Retry 가 전체 Thread 를 소진한다
EDU-ONL-08 — Optimistic Lock 동시 수정	F.19 배포 도중 Error Rate 가 증가한다
EDU-ONL-09 — 업무 Transaction 과 Audit	F.20 운영자가 결과를 수동 확정한다

본문

CPF 개발자 매뉴얼

기준 Repository: https://github.com/freeangelsun/202412_01_CPF

기준 Branch: master

기준 Commit: 77db10ad9aff44ee422795080fb2e96b364c9d65 (08_01)

기준일: 2026-08-07 Asia/Seoul

항목	내용
주 독자	온라인·연계 업무 개발자, Java/Spring 경험이 많지 않은 신규 개발자
이 문서로 완료할 일	PAY 예제를 따라 신규 업무 Domain 을 생성하고 Query·Command·DB·Message·외부 연계·보안·Test·ADM·운영 인계까지 만든다.
읽는 방식	처음 접하는 독자는 앞에서부터 실습 순서로 읽고, 숙련자는 장별 판단표와 참조 경로를 사용한다.
설명 원칙	제품 기능은 사용할 수 있는 상태로 설명한다. 실제 Source·SQL·API·Config·Frontend·Script·Test 의 이름과 경계를 유지한다.

PAY 00 00 00

00 0000 000 000 Source-SQL-API-Test-ADM-00 0000 00000.

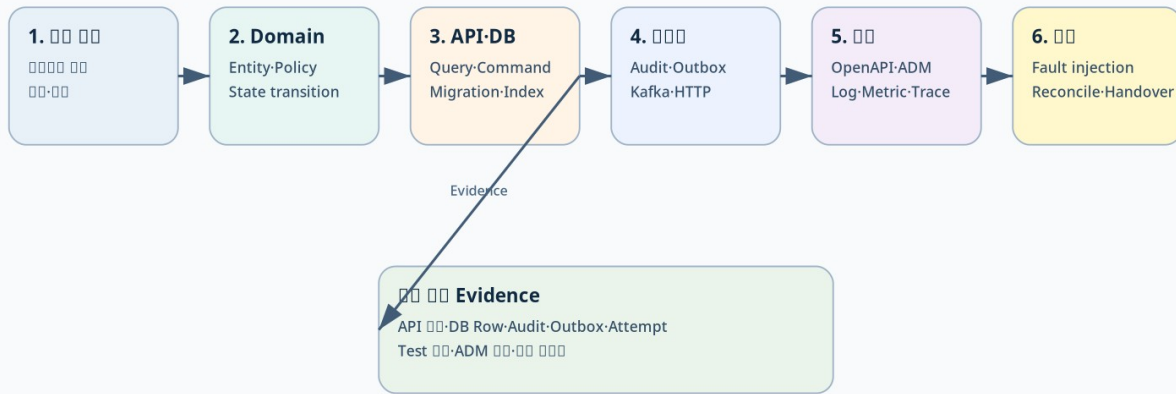


그림 - PAY 예제 학습 여정

1. 학습 목표와 준비

이 매뉴얼은 CPF 내부 구현을 설명하는 Reference 가 아니다. 예제 업무 PAY 를 책의 한 프로젝트처럼 따라 하며 다음 결과를 만든다.

- 결제수단 목록·상세 Query API
- 결제수단 등록·정지·재개 Command API
- 업무 Row·History·Audit·Idempotency·Outbox
- Kafka 후속 Event 와 외부기관 HTTP Attempt
- 응답 유실 UNKNOWN_RESULT 대사
- OpenAPI·Generated Client·ADM 연동 계약
- Unit·Contract·Integration·Fault Test
- 운영 인계표와 복구 판정

1.1 선수 지식

Java Record 와 Class, HTTP Method 와 JSON, SQL 의 PK·Index·Transaction, Git diff 를 알면 시작할 수 있다. Spring 의 세부 AutoConfiguration 을 몰라도 예제를 수행할 수 있도록 명령과 결과를 함께 제시한다.

1.2 예제 업무

고객은 여러 결제수단을 등록한다. 결제수단 상태는 PENDING, ACTIVE, SUSPENDED, CLOSED 다. 등록 뒤 외부기관에 계정을 등록하며 응답을 받지 못하면 UNKNOWN_RESULT 로 보존한다.

업무 규칙	내용
상태 Owner	PAY Domain
조회 권한	PAY_METHOD_READ + 고객 Data Scope
상태 변경 권한	PAY_METHOD_WRITE
정지 승인	고위험 결제수단은 Approval 필요
동시성	version Optimistic Lock
중복	Idempotency-Key + Request Hash
외부 대사 Key	operationId, institutionRequestId
감사	Actor·Reason·Before/After·Operation

2. 개발 환경과 기준 확인

Repository Root 를 D:\WORK_CPF\202412_01_CPF 로 가정한다.

```

$repo='D:\WORK_CPF\202412_01_CPF'
git -C $repo remote -v
git -C $repo branch --show-current
git -C $repo fetch origin master
git -C $repo rev-parse HEAD
git -C $repo rev-parse origin/master
  
```

```
git -C $repo status --short
java -version
& (Join-Path $repo 'gradlew.bat') --version
```

정상 결과:

- Remote 가 origin 과 CPF Repository 를 가리킨다.
- Branch 가 작업 기준 Branch 와 일치한다.
- Working Tree 의 기존 변경을 확인했다.
- Java·Gradle Version 은 gradle/cpf-stack.properties, Wrapper, BOM 기준과 맞는다.

기존 변경이 있으면 삭제·초기화하지 않는다. 새 Domain 파일과 기존 변경의 경로가 겹치는지 먼저 확인한다.

3. Profile 과 Capability 선택

PAY 는 Browser 와 외부 API 가 있고 권한·감사가 필요하므로 secure-api 또는 browser-bff 를 시작 Profile 로 선택한다. 후속 Event 와 외부 기관 HTTP 를 위해 Kafka·HTTP Capability 를 추가한다.

선택	값	이유
Profile	secure-api	인증·권한·Audit 가 있는 REST API
Data	JDBC 또는 MyBatis	업무 Row·Ledger·Audit 저장
Messaging	Kafka + reliability	Outbox·Inbox·DLQ
Integration	HTTP + resilience	외부기관 등록과 Timeout Budget
File	Attachment	신분 확인 자료가 필요한 경우
Security	Resource Server·Service Identity·Secret	사용자와 서비스 인증 분리
Operations	Observability·Runtime Control	Trace·Health·운영 조치

선택 결과는 Generator Manifest 와 resolved-starter-lock.json 에서 확인한다.

4. Generator Dry Run

Repository 의 Generator 명령은 실제 Script Parameter 를 확인한 뒤 사용한다. 다음은 표준 입력 예다.

```
$repo='D:\WORK_CPF\202412_01_CPF'
pwsh -NoProfile -ExecutionPolicy Bypass -File (Join-Path $repo 'cpf-tools\generator\create-domain.ps1') `
  -DomainName payment `
  -SystemCode PAY `
  -BasePackage com.customer.pay `
  -Profile secure-api `
  -DatabaseVendor postgresql `
  -DryRun
```

Dry Run 에서 다음을 읽는다.

1. 생성 Module·Package·Table Prefix.
2. Profile 이 해석한 Leaf Starter.
3. DB Vendor Pack 과 Migration 경로.
4. 기존 파일 충돌.
5. Generated 영역과 고객 수정 영역.
6. OpenAPI·Test·Manifest 생성 대상.

Dry Run 결과가 승인되면 같은 입력으로 적용하되 -DryRun 만 제거한다. 생성 경로를 임의 복사해 Package 이름만 바꾸지 않는다.

5. PAY Source 구조

```
pay-domain/
├── src/main/java/com/customer/pay/
│   ├── api/
│   │   ├── PayMethodQueryController.java
│   │   └── PayMethodCommandController.java
│   ├── dto/
│   ├── application/
│   │   ├── query/PayMethodQueryService.java
│   │   └── command/PayMethodCommandService.java
│   ├── port/
│   ├── domain/
│   │   ├── PayMethod.java
│   │   ├── PayMethodStatus.java
│   │   └── PayMethodPolicy.java
│   ├── persistence/
│   │   ├── PayMethodRepository.java
│   │   ├── PayIdempotencyRepository.java
│   │   └── PayAttemptRepository.java
│   ├── messaging/
│   └── integration/
├── src/main/resources/
│   ├── db/{mariadb,postgresql,oracle}/
│   ├── cpf-sql/pay/
└── src/test/
```

5.1 계층별 책임

계층	담당	놓지 않는 것
API	HTTP, 입력 형식, 인증 Context	DB Query, 상태 전이 규칙
Application	Use Case, Transaction, Permission, Idempotency	Provider SDK Type
Domain	상태·금액·정책 불변식	HTTP·DB·Kafka 세부 구현
Persistence	Query·Version·Ledger·Migration	화면 권한 판정
Adapter	HTTP·Kafka·SFTP 등 Provider 연결	업무 상태 Owner 역할

6. Domain 상태 모델

```
public enum PayMethodStatus {
    PENDING,
    ACTIVE,
    SUSPENDED,
    CLOSED
}

public final class PayMethod {
    private final String methodId;
    private PayMethodStatus status;
    private long version;

    public void suspend(long expectedVersion) {
        requireVersion(expectedVersion);
        if (status != PayMethodStatus.ACTIVE) {
            throw new IllegalStateException("ACTIVE 상태만 정지할 수 있습니다.");
        }
        status = PayMethodStatus.SUSPENDED;
        version++;
    }

    public void resume(long expectedVersion) {
        requireVersion(expectedVersion);
        if (status != PayMethodStatus.SUSPENDED) {
            throw new IllegalStateException("SUSPENDED 상태만 재개할 수 있습니다.");
        }
        status = PayMethodStatus.ACTIVE;
        version++;
    }

    private void requireVersion(long expectedVersion) {
        if (version != expectedVersion) {
            throw new PayVersionConflictException(methodId, expectedVersion, version);
        }
    }
}
```

Domain Test 는 Spring Context 없이 실행한다.

```
@Test
void active_method_can_be_suspended() {
    PayMethod method = fixture(PayMethodStatus.ACTIVE, 3L);
    method.suspend(3L);
    assertThat(method.status()).isEqualTo(PayMethodStatus.SUSPENDED);
    assertThat(method.version()).isEqualTo(4L);
}
```

금지 전이와 Version 충돌도 Test 한다.

7. DB Schema 와 Migration

7.1 논리 Table

Table	목적	핵심 Key
PAY_METHOD	결제수단 현재 상태	METHOD_ID, VERSION
PAY_METHOD_HISTORY	상태 변경 이력	HISTORY_ID, METHOD_ID
PAY_IDEMPOTENCY	요청 Key·Hash·결과 Snapshot	IDEMPOTENCY_KEY
PAY_OUTBOX	Commit 뒤 발행할 Event	EVENT_ID, STATUS
PAY_INSTITUTION_ATTEMPT	외부기관 요청·응답·불명 결과	ATTEMPT_ID, OPERATION_ID
PAY_AUDIT	Actor·Reason·Before/After	AUDIT_ID, OPERATION_ID

7.2 PostgreSQL 예제

```
create table pay_method (
    method_id varchar(40) primary key,
    customer_id varchar(40) not null,
    method_type varchar(30) not null,
    masked_account varchar(120) not null,
    status varchar(20) not null,
```

```

    version bigint not null,
    created_at timestamp with time zone not null,
    updated_at timestamp with time zone not null,
    constraint ck_pay_method_status
        check (status in ('PENDING', 'ACTIVE', 'SUSPENDED', 'CLOSED'))
);

create index ix_pay_method_customer_status
on pay_method(customer_id, status, method_id);

```

MariaDB 와 Oracle Script 는 같은 Column 의미, 상태 Constraint, Index 목적을 유지하되 Vendor 문법을 분리한다. 하나의 Vendor 만 변경하지 않는다.

7.3 Optimistic Update

```

update pay_method
set status = :target_status,
    version = version + 1,
    updated_at = :now
where method_id = :method_id
and version = :expected_version

```

영향 건수가 0 이면 Row 없음과 Version 충돌을 구분해 재조회한다.

8. Query API

8.1 DTO

```

public record PayMethodCriteria(
    String customerId,
    PayMethodStatus status,
    int size,
    String cursor,
    String sort) {

    public PayMethodCriteria {
        if (customerId == null || customerId.isBlank()) {
            throw new IllegalArgumentException("customerId is required");
        }
        if (size < 1 || size > 200) {
            throw new IllegalArgumentException("size must be between 1 and 200");
        }
    }
}

public record PayMethodView(
    String methodId,
    String maskedAccount,
    PayMethodStatus status,
    long version,
    Instant updatedAt) {}

```

8.2 Controller

```

@GetMapping("/api/pay/methods")
PayMethodPage findMethods(
    @Valid PayMethodCriteria criteria,
    CpfSecurityContext actor) {
    return queryService.find(criteria, actor);
}

```

Query Service 가 할 일:

7. Actor 의 고객 Data Scope 를 Criteria 에 적용한다.
8. Sort whitelist 를 검증한다.
9. 안정된 Cursor Key 를 사용한다.
10. Repository 결과를 Masking 정책으로 변환한다.
11. 기준시각과 Source Owner 를 응답에 포함한다.

8.3 Negative Test

- 허용하지 않은 Sort 이름.
- 최대 Page 크기 초과.
- 다른 고객의 customerId 조회.
- 잘못된 Cursor Encoding.
- 상세 조회 Row 의 Masking 누락.
- Owner Query Timeout.

9. Command API

Command 생명주기

Command의 생명주기는 다음과 같다.

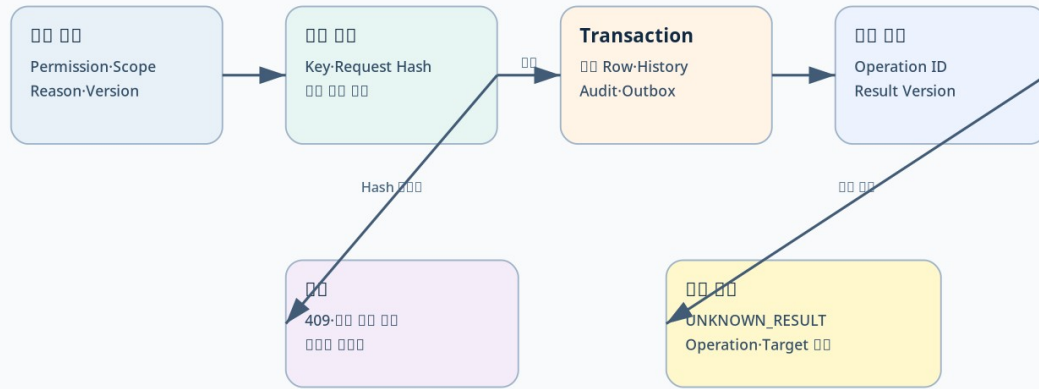


그림 - Command 생명주기

9.1 입력

```

public record ChangePayMethodStatusCommand(
    String methodId,
    PayMethodStatus targetStatus,
    long expectedVersion,
    String idempotencyKey,
    String reason) {}
  
```

reason은 “처리”, “확인”처럼 의미 없는 값이 아니라 업무 판단 근거를 남긴다. 길이·제어문자·민감정보 포함 여부를 검증한다.

9.2 Application Service 예제

```

@Transactional
public PayOperationResult suspend(
    ChangePayMethodStatusCommand command,
    CpfSecurityContext actor) {

    permission.require(actor, "PAY_METHOD_WRITE", command.methodId());
    String requestHash = requestHasher.hash(command);

    var replay = idempotency.find(command.idempotencyKey());
    if (replay.isPresent()) {
        return replay.get().sameHashOrThrow(requestHash).result();
    }

    PayMethod method = repository.getRequired(command.methodId());
    PayMethodSnapshot before = method.snapshot();
    method.suspend(command.expectedVersion());

    String operationId = operationIds.next("PAY-SUSPEND");
    repository.update(method);
    history.append(method, operationId);
    audit.append(actor, command.reason(), before, method.snapshot(), operationId);
    outbox.append(PayMethodSuspended.from(method, operationId));

    PayOperationResult result = PayOperationResult.succeeded(operationId, method);
    idempotency.save(command.idempotencyKey(), requestHash, result);
    return result;
}
  
```

Transaction 안에는 업무 Row·History·Audit·Outbox·Idempotency Result를 넣는다. 외부기관 HTTP 호출은 이 Transaction 안에서 실행하지 않는다.

9.3 API 응답

```

{
  "operationId": "PAY-SUSPEND-20260806-000041",
  "methodId": "PM-000123",
  "previousStatus": "ACTIVE",
  "status": "SUSPENDED",
  "resultVersion": 8,
  "replayed": false,
  "auditId": "AUD-000991"
}
  
```

응답을 잃은 Client는 같은 Idempotency Key로 재조회한다. 같은 Key에 다른 Payload를 보내면 Conflict다.

10. 오류 응답 계약

HTTP	오류	의미	Client 행동
400	CPF - VALIDATION	형식·범위 오류	입력 수정
401	CPF - AUTHENTICATION	인증 없음·만료	재인증
403	CPF - FORBIDDEN	Permission·Data Scope 거부	권한 요청 또는 대상 축소
404	PAY - METHOD - NOT - FOUND	대상 없음	ID 재확인
409	PAY - VERSION - CONFLICT	최신 Version 과 불일치	최신 Row 비교 후 새 요청
409	CPF - IDEMPOTENCY - CONFLICT	같은 Key·다른 Hash	새 Key 로 새 의도 제출
504	CPF - TIMEOUT	Dispatch 전 Timeout	상태 확인 후 재시도 판단
202	UNKNOWN_RESULT	Dispatch 뒤 결과 불명	Operation 조회·대사

오류 응답에는 Error Code, Message, Transaction ID, Operation ID, Retry 가능 여부를 포함한다. Stack Trace 와 Secret 은 노출하지 않는다.

11. Outbox·Kafka·Inbox

11.1 Event

```
public record PayMethodSuspended(
    String eventId,
    String operationId,
    String methodId,
    long version,
    Instant occurredAt) {}
```

11.2 Producer 흐름

12. 업무 Transaction 에서 Outbox Row 를 READY 로 저장한다.
13. Publisher 가 Lease 를 확보한다.
14. Kafka Record Key 로 methodId 또는 순서가 필요한 업무 Key 를 사용한다.
15. Publish 결과와 Attempt 를 저장한다.
16. Confirm 뒤 Outbox 를 PUBLISHED 로 바꾼다.
17. Process 가 종료되면 Lease 만료 뒤 다른 Instance 가 재개한다.

11.3 Consumer 흐름

18. Event ID·Consumer ID 로 Inbox 를 확보한다.
19. 이미 SUCCEEDED 면 ACK 하고 업무를 반복하지 않는다.
20. 업무 Transaction 을 수행한다.
21. Inbox 결과를 SUCCEEDED 로 저장한다.
22. Commit 뒤 ACK 한다.
23. Poison 은 DLQ 로 보내고 운영 Replay 를 사용한다.

11.4 Fault Test

- Outbox 저장 뒤 Publisher 종료.
- Broker Publish 뒤 Confirm 저장 전 종료.
- Consumer 업무 Commit 뒤 ACK 전 종료.
- 같은 Event 중복 Delivery.
- Schema Version 불일치.
- DLQ Replay 중 같은 Payload 중복.

12. 외부기관 HTTP 와 결과 불명

12.1 Port

```
public interface PayInstitutionPort {
    InstitutionRegistrationResult register(
        InstitutionRegistrationRequest request,
        CpfDeadline deadline,
        CpfServiceIdentity identity);

    InstitutionRegistrationStatus findStatus(String institutionRequestId);
}
```

12.2 Attempt 저장

외부 전송 전에 다음을 저장한다.

- Operation ID
- Attempt 번호

- Target ID 와 URI 식별자
- 업무 Key
- Payload Hash
- Idempotency Key
- Deadline
- Request 시각

12.3 상태 분류

실패 지점	상태	행동
DNS·Connection 전 실패	FAILED	정책이 허용하면 새 Attempt
Request 전송 중 연결 종료	UNKNOWN_RESULT 가능	Target 조회로 접수 여부 확인
Target 4xx	FAILED	입력·권한 수정, 동일 Blind Retry 금지
Target 5xx 명시	정책에 따른 FAILED	역등성·조회 경로 확인 후 Retry
Target 처리 후 응답 유실	UNKNOWN_RESULT	기관 접수 ID·업무 Key 대사
여러 기관 중 일부 성공	PARTIAL	성공 기관 보존, 실패 기관만 조치

12.4 WireMock Fault 예

```
stubFor(post(urlEqualTo("/institution/methods"))
    .willReturn(aResponse().withFixedDelay(8_000).withStatus(201)));
```

Client Timeout 을 3 초로 두면 Target 은 접수했지만 Client 는 응답을 받지 못하는 상황을 재현할 수 있다. Test 는 Operation 을 UNKNOWN_RESULT 로 저장하고 findStatus 대사 뒤 SUCCEEDED 로 종결되는지 확인한다.

13. Local·Remote 동일 계약

```
public interface PayMethodOwnerPort {
    PayMethodPage find(PayMethodCriteria criteria, CpfSecurityContext actor);
    PayOperationResult suspend(ChangePayMethodStatusCommand command, CpfSecurityContext actor);
}
```

Local Adapter 와 Remote Adapter 는 같은 Port 를 구현한다.

항목	Local	Remote
DTO·Validation	동일	동일
업무 Error	동일	HTTP/Serialization Mapping 추가
Permission	동일 의미	Identity·Audience 전달
Transaction	Same-JVM 경계	분리 Transaction, Attempt 필요
Timeout	내부 Deadline	Network·Queue 포함
Test	Port Contract Fixture	같은 Fixture + Network Fault

업무 코드는 Adapter 구현을 직접 참조하지 않는다.

14. OpenAPI WebMVC

06_06 이후 cpf.openapi.webmvc 설정은 Web API 의 OpenAPI Snapshot 과 운영 상태를 제공한다.

Property	Type	Default	설명
cpf.openapi.webmvc.enabled	Boolean	true	OpenAPI 기능 활성화
cpf.openapi.webmvc.api-docs-enabled	Boolean	false	API Docs Endpoint 노출
cpf.openapi.webmvc.management-enabled	Boolean	true	관리 기능 노출
cpf.openapi.webmvc.api-docs-path	Path	/v3/api-docs	.. 없는 절대 Path
cpf.openapi.webmvc.title	String	CPF API	문서 제목
cpf.openapi.webmvc.version	String	unspecified	API Version
cpf.openapi.webmvc.instance-id	String	local	Snapshot Instance
cpf.openapi.webmvc.minimum-refresh-interval	Duration	1s	Refresh 최소 간격

Operation ID 는 Controller, OpenAPI, Generated Client, ADM Route, Permission, Audit 를 연결한다. 중복 Operation ID 와 Schema Drift 는 Build Gate 에서 차단한다.

15. Security·Data Scope·Masking

15.1 사용자와 서비스 Identity

- Browser/User 요청: 사용자 Subject, Role, Session, MFA 상태.
- Service 호출: Service ID, Audience, Credential Version.
- 사용자 Token 을 서비스 장기 Credential 로 재사용하지 않는다.

15.2 Permission 판정

```
permission.require(actor, "PAY_METHOD_WRITE", methodId);
dataScope.requireCustomer(actor, customerId);
```

Frontend Button 숨김만으로 보호하지 않는다. Controller 와 Application Service 에서 다시 판정한다.

15.3 Field 암호화

Field 마다 분류, Masking, 검색 필요 여부, Key Version 을 정한다. 검색이 필요한 값은 암호문과 Search Token 을 구분한다. 복호화는 Actor·Reason 을 감사에 남긴다.

16. Attachment 예제

16.1 처리 순서

24. Upload Session 발급.
25. Size·MIME·File Name 검증.
26. Temp Object 저장.
27. Checksum 계산.
28. Malware Scan.
29. 성공 시 Final Object Key 로 전환.
30. Metadata·Scan Result·Audit 저장.
31. Download Permission 과 Retention 적용.

16.2 상태

UPLOADING → RECEIVED → SCANNING → AVAILABLE 이며 Scan 실패는 QUARANTINED 다. Object Store 저장 성공과 Metadata Commit 사이에 Process 가 종료되면 Checksum·Object Key 로 대사한다.

17. 데이터 품질 격리·정정·Replay

CpfDataQualityOperations 를 사용해 Rule Version 과 위반을 기록한다. ERROR 또는 CRITICAL 위반은 정상 원장 반영을 중단하고 Quarantine Item 을 만든다.

ADM 통합 정정 흐름은 다음 계약을 사용한다.

32. Quarantine ID, Expected Version, Reason, Corrected JSON 으로 승인 요청.
33. Server 가 정정 Payload Snapshot 을 고정.
34. 승인 결정.
35. 실행 시 Approval ID 와 Reason 만 전달.
36. Owner 가 Version 과 Snapshot 을 다시 검증.
37. 결과를 CORRECTED 또는 오류 상태로 기록.
38. Replay 성공 시 REPLAYED 로 종결.

Client 가 approved=true 를 Body 에 넣어 승인을 우회하는 방식은 사용하지 않는다.

18. 시간·Deadline

UTC 와 업무 Zone 을 분리한다. 한국 영업일은 Asia/Seoul 을 사용하되 DB 에는 UTC 시각과 업무일자를 각각 저장한다. Timeout 은 wall clock 차감보다 Deadline·monotonic time 을 사용한다.

Test 에서는 고정 Clock 으로 자정, 영업일 전환, 만료, Clock rollback 을 재현한다.

19. Test 전략

층	정상 Test	오류·경계	복구 Test
Domain	허용 상태 전이	금지 전이·금액 경계	불변식 유지
Application	Use Case·Transaction	Version·Permission·Idempotency	Replay·Compensation
Persistence	CRUD·Index·Migration	Deadlock·Constraint·Vendor 차이	Rollback·Forward recovery
API	Contract·OpenAPI	400·401·403·404·409	같은 Key 결과 조회

층	정상 Test	오류·경계	복구 Test
Messaging	Publish·Consume	Duplicate·Poison·Kill	Inbox·DLQ Replay
HTTP	2xx·4xx·5xx	Timeout·TLS·응답 유실	Attempt Reconcile
Browser	Field·Button·상태	Permission·Timeout	Retry·Reconcile UI

19.1 실행 예

```
$repo='D:\WORK_CPF\202412_01_CPF'
& (Join-Path $repo 'gradlew.bat') :pay-domain:test --tests '*PayMethod*' --stacktrace
```

실제 Module 이름은 생성 Manifest 를 따른다.

20. Fault Injection 실습

실습 A — 중복 요청

- 같은 Idempotency Key 와 같은 Body 를 두 번 보낸다.
- 두 번째 응답의 `replayed=true` 를 확인한다.
- 업무 Row·History·Audit·Outbox 가 한 번만 생성됐는지 확인한다.

실습 B — Version 충돌

- Version 7 Row 를 두 사용자가 읽는다.
- A 가 정지해 Version 8 을 만든다.
- B 가 Expected Version 7 로 재개한다.
- 409 와 최신 Version 8 을 확인한다.

실습 C — Consumer Kill

- Consumer 가 업무 Commit 뒤 ACK 전에 종료되도록 Fault Point 를 둔다.
- Broker 가 같은 Event 를 재전달한다.
- Inbox 가 기존 결과를 반환하고 업무 Row 가 중복되지 않는지 확인한다.

실습 D — HTTP 응답 유실

- Target 은 접수하지만 Client Timeout 을 발생시킨다.
- Attempt 가 UNKNOWN_RESULT 인지 확인한다.
- Target Status API 로 결과를 조회한다.
- 같은 원본 Attempt 를 수정하지 않고 Reconciliation Decision 을 추가한다.

21. ADM 확인

개발자는 다음 Route 에서 결과를 찾을 수 있어야 한다.

- /transactions: 온라인 거래 정의와 상태.
- /transactionGroups: Transaction·Trace Timeline.
- /logs: 요청·응답·Error.
- /auditLogs: Actor·Reason·Before/After.
- /reliability: Outbox·Inbox·Idempotency·Unknown.
- /incidents: 미종결 오류와 운영 조치.
- /integrationClosure: 데이터 품질 정정 승인·Replay, Webhook DLQ, Crypto·Time 상태.
- /openApiOperations: OpenAPI 상태와 Refresh.

22. 운영 인계표

항목	PAY 예
Owner	PAY Domain
Consumer	Browser·ADM·Kafka Consumer·기관 Adapter
API	/api/pay/methods, 상태 변경 Command
Table	PAY_METHOD, History, Idempotency, Outbox, Attempt, Audit
Topic	pay.method.changed.v1
외부 Target	기관 등록 API 와 Status 조회 API
상태	PENDING·ACTIVE·SUSPENDED·CLOSED·UNKNOWN_RESULT
대사 Key	Operation ID·Institution Request ID·Payload Hash
Alert	Outbox Lag·Unknown Age·DLQ·HTTP Error Rate

항목	PAY 예
복구	Replay · Reconcile · Compensation · Rollback 조건
배포 영향	Migration · Config · Secret · Restart 여부
Rollback	이전 Artifact · Schema 호환 · Event Version

23. 고객 업무로 바꾸는 부분

고객이 바꾸는 것:

- Domain 이름과 상태.
- Field 와 Validation.
- Permission · Data Scope.
- 외부기관 SLA 와 대사 규칙.
- Event Schema 와 업무 Key.
- Alert 임계치와 승인 정책.

CPF 계약으로 유지할 것:

- Context · Error · 식별자.
- Public API/SPI/Internal 경계.
- Idempotency · Attempt · Audit 구조.
- Generated Client 와 OpenAPI Source.
- Migration · 3 Vendor · Test · 운영 인계 형식.

24. 개발자 자체 검수

53. 업무 결과와 Owner 를 한 문장으로 설명할 수 있는가?
54. Query 와 Command 가 분리됐는가?
55. Public 계약에 Provider SDK Type 이 없는가?
56. 상태 변경에 Version · Reason · Idempotency 가 있는가?
57. 업무 Row · History · Audit · Outbox 가 같은 Transaction 인가?
58. 응답 유실을 FAILED 와 구분하는가?
59. Local · Remote 가 같은 Contract Fixture 를 통과하는가?
60. 3 개 DB Vendor 의 논리 계약이 같은가?
61. OpenAPI · Generated Client · ADM Route 가 연결되는가?
62. 정상 · 중복 · 동시성 · Timeout · Process Kill · 부분 실패를 재현했는가?
63. ADM 에서 Operation 과 Audit 를 찾을 수 있는가?
64. 운영 인계표만 읽고 다음 담당자가 복구할 수 있는가?

PAY Golden Journey

□□□□ □□ □□□ ADM □□□□

1

API Validation

Permission · Data Scope · Idempotency · Expected Version

2

Transaction

□□ Row · History · Audit · Outbox □□ □□

3

Async

Publisher · Broker · Inbox · DLQ

4

External Attempt

Dispatch · Timeout · UNKNOWN_RESULT

5

Reconciliation

Target Status · □□ □□ · □□ □□

6

Operations

Trace · Audit · Incident · Recovery

그림 - PAY Golden Journey

25. 08_01 R6 계약 반영: 승인 정정과 역등성

08_01 기준으로 ADM 통합 정정의 브라우저 역등성 원장이 v3 로 변경되었다. expectedVersion 은 Safe Integer 이면서 1 이상이어야 하고, Draft 는 Unicode NFC 정규화와 Key 정렬 뒤 SHA-256 Fingerprint 를 만든다. 같은 Fingerprint·Generation 은 탭이 달라도 결정적인 Idempotency Key 를 사용한다. Pending 원장은 24 시간, Confirmed 원장은 7 일 TTL 을 가지며 최대 64 개 Entry 를 유지한다. 고객 업무에서 같은 패턴을 적용할 때는 브라우저 저장소를 성공 판정의 정보으로 사용하지 말고 서버 Idempotency 원장과 함께 사용한다.

정정 승인 API 는 expectedVersion >= 1, idempotencyKey 8..128, reason 8..500, 비어 있지 않은 corrected 를 요구한다. Replay 도 expectedVersion·idempotencyKey·reason 을 Body 로 받아 재검증 자체가 중복 실행되지 않도록 한다. 이 계약은 AdmIntegrationClosureController 와 Frontend integrationClosureIdempotency.ts 를 기준으로 한다.

부록 A. PAY 예제 전체 연결 Source

이 부록은 앞 장의 부분 코드를 한 흐름으로 연결한다. Package 와 Class 이름은 고객 업무 예제이며, CPF 의 Public API·SPI·Generated 영역을 임의 수정하지 않는다.

A.1 요청·응답 DTO

```
package com.customer.pay.api.dto;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Positive;
import java.time.Instant;

public record RegisterPayMethodRequest(
    @NotBlank String customerId,
    @NotBlank String providerCode,
    @NotBlank String accountToken,
    @NotBlank String reason) {}

public record ChangePayMethodStatusRequest(
    @Positive long expectedVersion,
    @NotBlank String reason) {}

public record PayMethodResponse(
    String methodId,
    String customerId,
    String providerCode,
    String maskedAccount,
    PayMethodStatus status,
    long version,
    String operationId,
    Instant updatedAt) {}
```

A.2 Command Controller

```
package com.customer.pay.api;

import com.customer.pay.api.dto.ChangePayMethodStatusRequest;
import com.customer.pay.api.dto.PayMethodResponse;
import com.customer.pay.api.dto.RegisterPayMethodRequest;
import com.customer.pay.application.PayMethodCommandService;
import jakarta.validation.Valid;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/pay/methods")
public final class PayMethodCommandController {
    private final PayMethodCommandService service;

    public PayMethodCommandController(PayMethodCommandService service) {
        this.service = service;
    }

    @PostMapping
    public ResponseEntity<PayMethodResponse> register(
        @RequestHeader("Idempotency-Key") String idempotencyKey,
        @Valid @RequestBody RegisterPayMethodRequest request) {
        return ResponseEntity.ok(service.register(idempotencyKey, request));
    }

    @PostMapping("/{methodId}/suspend")
    public ResponseEntity<PayMethodResponse> suspend(
        @PathVariable String methodId,
        @RequestHeader("Idempotency-Key") String idempotencyKey,
        @Valid @RequestBody ChangePayMethodStatusRequest request) {
        return ResponseEntity.ok(service.suspend(methodId, idempotencyKey, request));
    }

    @PostMapping("/{methodId}/resume")
    public ResponseEntity<PayMethodResponse> resume(
        @PathVariable String methodId,
        @RequestHeader("Idempotency-Key") String idempotencyKey,
        @Valid @RequestBody ChangePayMethodStatusRequest request) {
        return ResponseEntity.ok(service.resume(methodId, idempotencyKey, request));
    }
}
```

A.3 Application Service 의 처리 순서

```
@Transactional
public PayMethodResponse suspend(
    String methodId,
    String idempotencyKey,
    ChangePayMethodStatusRequest request) {
    CommandFingerprint fingerprint = fingerprintFactory.create(
        "PAY_METHOD_SUSPEND", methodId, request);

    return idempotencyRepository.execute(
        idempotencyKey,
        fingerprint,
        () -> {
            actorPermission.require("PAY_METHOD_WRITE", methodId);
            PayMethod current = repository.getForUpdate(methodId);
            PayMethod before = current.snapshot();
            current.suspend(request.expectedVersion());
            repository.update(current);
            historyRepository.append(before, current, request.reason());
            auditRepository.append(AuditEntry.of(
                operationContext.operationId(),
                actorContext.actorId(),
                "PAY_METHOD_SUSPEND",
                methodId,
                request.reason(),
                before,
                current));
            outboxRepository.append(PayMethodChangedEvent.from(current));
            return mapper.toResponse(current, operationContext.operationId());
        });
}
```

처리 순서를 바꾸지 않는다. Permission 과 Version 검증 전에 외부 호출을 보내지 않고, 업무 Row·History·Audit·Outbox 가 같은 Transaction 에서 Commit 되도록 한다.

A.4 JDBC Optimistic Update

```
UPDATE PAY_METHOD
SET STATUS = :status,
    VERSION = VERSION + 1,
    UPDATED_AT = :updatedAt,
    UPDATED_BY = :updatedBy
WHERE METHOD_ID = :methodId
AND VERSION = :expectedVersion;
```

Update Count 가 0 이면 존재하지 않음과 Version 충돌을 구분해 조회한다. 현재 Version 을 응답에 포함하되 민감 Field 는 Masking 한다.

A.5 Outbox Publisher

```
public int publishBatch(int limit) {
    List<OutboxRecord> claimed = outboxRepository.claim(
        publisherId, fencingTokenProvider.current(), limit);
    int published = 0;
    for (OutboxRecord record : claimed) {
        try {
            brokerPort.publish(record.topic(), record.key(), record.payload(), record.headers());
            outboxRepository.markPublished(record.id(), record.version(), clock.instant());
            published++;
        } catch (RetryableBrokerException retryable) {
            outboxRepository.markRetry(record.id(), retryable.code(), retryPolicy.next(record.attempt()));
        } catch (Exception permanent) {
            outboxRepository.moveToDlq(record.id(), permanent.getClass().getSimpleName());
        }
    }
    return published;
}
```

A.6 Inbox Consumer

```
@Transactional
public void onPayMethodChanged(PayMethodChangedEvent event) {
    InboxResult existing = inboxRepository.find(consumerId, event.eventId());
    if (existing != null) {
        return;
    }
    notificationProjection.apply(event);
    inboxRepository.insert(consumerId, event.eventId(), event.payloadHash(), clock.instant());
}
```

업무 반영과 Inbox 기록이 같은 Transaction 이어야 한다. ACK 는 Transaction Commit 이후 수행한다.

A.7 외부기관 Attempt 와 결과 불명

```
public InstitutionRegistrationResult registerAtInstitution(PayMethod method) {
    ExternalAttempt attempt = attemptRepository.start(
        operationContext.operationId(),
        method.methodId(),
        payloadHasher.hash(method),
        deadline.remaining());
    try {
        InstitutionResponse response = institutionPort.register(attempt.requestId(), method);
        return attemptRepository.complete(attempt.id(), response.trackingId(), response.status());
    } catch (ReadTimeoutAfterDispatch timeout) {
        attemptRepository.markUnknown(attempt.id(), timeout.getMessage());
        return InstitutionRegistrationResult.unknown(attempt.operationId());
    }
}
```

```

    } catch (ConnectFailureBeforeDispatch notSent) {
        attemptRepository.markFailed(attempt.id(), notSent.getMessage());
        throw notSent;
    }
}

```

ReadTimeoutAfterDispatch 는 실패로 확정하지 않는다. 기관 Tracking ID 또는 업무 Key 로 상태조회한 뒤 Reconciliation Decision 을 새 Row 로 남긴다.

A.8 Error Response

```

{
  "code": "PAY_VERSION_CONFLICT",
  "message": "다른 사용자가 먼저 변경했습니다.",
  "transactionId": "TX-20260806-000001",
  "operationId": "OP-20260806-000031",
  "details": {
    "methodId": "PM-1001",
    "expectedVersion": 7,
    "currentVersion": 8
  }
}

```

A.9 통합 Test 시나리오

```

@Test
void response_loss_is_reconciled_without_duplicate_registration() {
    String key = "idem-pay-register-001";
    institution.stubAcceptedThenTimeout("REQ-001", "TRACK-777");

    PayMethodResponse first = client.register(key, request());
    assertThat(first.status()).isEqualTo(PayMethodStatus.UNKNOWN_RESULT);

    institution.stubStatus("TRACK-777", "COMPLETED");
    reconciliationJob.run(first.operationId());

    PayMethodResponse resolved = client.get(first.methodId());
    assertThat(resolved.status()).isEqualTo(PayMethodStatus.ACTIVE);
    assertThat(institution.registrationCount("REQ-001")).isEqualTo(1);
    assertThat(attemptRepository.history(first.operationId()))
        .extracting(ExternalAttempt::status)
        .containsExactly(AttemptStatus.UNKNOWN_RESULT, AttemptStatus.COMPLETED);
}

```

부록 B. 온라인·공통·외부 연계 EDU 45 개

온라인 EDU 는 요청 준비부터 API·DB·Message·외부 Attempt 의 정상 결과를 확인한 뒤, 중복·동시성·Timeout·응답 유실을 재현하고 복구한다. 적용할 때는 기준 Commit 의 Public API 와 Test 경로를 다시 대조한다.

EDU-ONL-01 — Generator Dry Run 과 충돌 검토

항목	수행 내용
학습 결과	Generator Dry Run 과 충돌 검토의 선택 기준과 정상·실패·복구 의미를 설명하고 직접 판정한다.
Repository 확인 위치	cpf-tools/generator 및 실제 Consumer·Test·Config
주요 입력	DomainName·SystemCode·Profile·DB Vendor
실행 순서	입력 Fixture 와 기준 상태 확인 → 정상 실행과 원장 변화 확인 → 로그·Trace·Audit 상호 대조 → 해당 EDU 의 장애 조건 재현 → 상태 Owner 기준 복구 → 동일 입력으로 재검증
정상 판정	생성 경로·Starter Lock·충돌 목록 확인
장애 재현	기존 파일 충돌·지원하지 않는 조합
복구 판정	Dry Run 결과를 수정하고 적용 전 diff 를 재검토
운영 확인	/configs
고객 업무 전환	예제 ID·상태·Permission·SLA 만 고객 값으로 바꾸고 Idempotency·Version·Audit·복구 계약은 유지

EDU-ONL-02 — secure-api Profile 조립

항목	수행 내용
학습 결과	secure-api Profile 조립의 선택 기준과 정상·실패·복구 의미를 설명하고 직접 판정한다.
Repository 확인 위치	cpf-starters/profiles/secure-api 및 실제 Consumer·Test·Config
주요 입력	Resource Server·Audit·Persistence 선택
실행 순서	입력 Fixture 와 기준 상태 확인 → 정상 실행과 원장 변화 확인 → 로그·Trace·Audit 상호 대조 → 해당 EDU 의 장애 조건 재현 → 상태 Owner 기준 복구 → 동일 입력으로 재검증
정상 판정	선택 Starter 만 Classpath·Health 에 나타남
장애 재현	선택하지 않은 Provider Bean 유입

항목	수행 내용
복구 판정	Resolved Lock 과 dependencyInsight 로 원인을 제거
운영 확인	/openApiOperations
고객 업무 전환	예제 ID · 상태 · Permission · SLA 만 고객 값으로 바꾸고 Idempotency · Version · Audit · 복구 계약은 유지

EDU-ONL-03 — 조건 검색과 Paging

항목	수행 내용
학습 결과	조건 검색과 Paging 의 선택 기준과 정상 · 실패 · 복구 의미를 설명하고 직접 판정한다.
Repository 확인 위치	cpf - reference 온라인 EDU 및 실제 Consumer · Test · Config
주요 입력	검색 Field · Page · Size · Sort
실행 순서	입력 Fixture 와 기준 상태 확인 → 정상 실행과 원장 변화 확인 → 로그 · Trace · Audit 상호 대조 → 해당 EDU 의 장애 조건 재현 → 상태 Owner 기준 복구 → 동일 입력으로 재검증
정상 판정	허용 Sort 와 Data Scope 가 적용된 Page 응답
장애 재현	허용되지 않은 Sort · 과대 Page
복구 판정	400 으로 차단하고 최대 Page Size 를 유지
운영 확인	/transactions
고객 업무 전환	예제 ID · 상태 · Permission · SLA 만 고객 값으로 바꾸고 Idempotency · Version · Audit · 복구 계약은 유지

EDU-ONL-04 — 상세 조회와 Masking

항목	수행 내용
학습 결과	상세 조회와 Masking 의 선택 기준과 정상 · 실패 · 복구 의미를 설명하고 직접 판정한다.
Repository 확인 위치	cpf - reference 온라인 EDU 및 실제 Consumer · Test · Config
주요 입력	Business ID · Actor · Data Scope
실행 순서	입력 Fixture 와 기준 상태 확인 → 정상 실행과 원장 변화 확인 → 로그 · Trace · Audit 상호 대조 → 해당 EDU 의 장애 조건 재현 → 상태 Owner 기준 복구 → 동일 입력으로 재검증
정상 판정	권한별 Masking 결과와 조회 Audit
장애 재현	타 조직 ID · 원문 민감값 조회
복구 판정	403 또는 Masked 응답, Audit 로 시도 확인
운영 확인	/auditLogs
고객 업무 전환	예제 ID · 상태 · Permission · SLA 만 고객 값으로 바꾸고 Idempotency · Version · Audit · 복구 계약은 유지

EDU-ONL-05 — Versioned 상태 변경

항목	수행 내용
학습 결과	Versioned 상태 변경의 선택 기준과 정상 · 실패 · 복구 의미를 설명하고 직접 판정한다.
Repository 확인 위치	cpf - reference 온라인 EDU 및 실제 Consumer · Test · Config
주요 입력	expectedVersion · reason · command body
실행 순서	입력 Fixture 와 기준 상태 확인 → 정상 실행과 원장 변화 확인 → 로그 · Trace · Audit 상호 대조 → 해당 EDU 의 장애 조건 재현 → 상태 Owner 기준 복구 → 동일 입력으로 재검증
정상 판정	상태 · Version · History · Audit 가 한 Transaction 으로 변경
장애 재현	오래된 Version
복구 판정	409 와 현재 Version 반환 후 사용자가 재판단
운영 확인	/transactionGroups
고객 업무 전환	예제 ID · 상태 · Permission · SLA 만 고객 값으로 바꾸고 Idempotency · Version · Audit · 복구 계약은 유지

EDU-ONL-06 — Idempotency 동일 요청 재호출

항목	수행 내용
학습 결과	Idempotency 동일 요청 재호출의 선택 기준과 정상·실패·복구 의미를 설명하고 직접 판정한다.
Repository 확인 위치	cpf-reference reliability EDU 및 실제 Consumer·Test·Config
주요 입력	Idempotency-Key·Request Hash
실행 순서	입력 Fixture 와 기준 상태 확인 → 정상 실행과 원장 변화 확인 → 로그·Trace·Audit 상호 대조 → 해당 EDU 의 장애 조건 재현 → 상태 Owner 기준 복구 → 동일 입력으로 재검증
정상 판정	원본 결과를 반환하고 부작용 1 회
장애 재현	응답 유실 뒤 같은 요청 재호출
복구 판정	원본 Operation 결과 조회, 새 Row 생성 금지
운영 확인	/reliability
고객 업무 전환	예제 ID·상태·Permission·SLA 만 고객 값으로 바꾸고 Idempotency·Version·Audit·복구 계약은 유지

EDU-ONL-07 — Idempotency Key Payload 충돌

항목	수행 내용
학습 결과	Idempotency Key Payload 충돌의 선택 기준과 정상·실패·복구 의미를 설명하고 직접 판정한다.
Repository 확인 위치	cpf-reference reliability EDU 및 실제 Consumer·Test·Config
주요 입력	같은 Key·다른 Payload
실행 순서	입력 Fixture 와 기준 상태 확인 → 정상 실행과 원장 변화 확인 → 로그·Trace·Audit 상호 대조 → 해당 EDU 의 장애 조건 재현 → 상태 Owner 기준 복구 → 동일 입력으로 재검증
정상 판정	409 또는 typed conflict
장애 재현	Key 재사용으로 다른 업무 실행 시도
복구 판정	원본 Hash 를 유지하고 새 Key 발급
운영 확인	/incidents
고객 업무 전환	예제 ID·상태·Permission·SLA 만 고객 값으로 바꾸고 Idempotency·Version·Audit·복구 계약은 유지

EDU-ONL-08 — Optimistic Lock 동시 수정

항목	수행 내용
학습 결과	Optimistic Lock 동시 수정의 선택 기준과 정상·실패·복구 의미를 설명하고 직접 판정한다.
Repository 확인 위치	cpf-reference concurrency EDU 및 실제 Consumer·Test·Config
주요 입력	두 사용자의 동일 Version
실행 순서	입력 Fixture 와 기준 상태 확인 → 정상 실행과 원장 변화 확인 → 로그·Trace·Audit 상호 대조 → 해당 EDU 의 장애 조건 재현 → 상태 Owner 기준 복구 → 동일 입력으로 재검증
정상 판정	한 요청 성공·다른 요청 409
장애 재현	Last-write-wins 로 덮어쓰기
복구 판정	최신 상태 재조회와 사유 재입력
운영 확인	/transactions
고객 업무 전환	예제 ID·상태·Permission·SLA 만 고객 값으로 바꾸고 Idempotency·Version·Audit·복구 계약은 유지

EDU-ONL-09 — 업무 Transaction 과 Audit

항목	수행 내용
학습 결과	업무 Transaction 과 Audit 의 선택 기준과 정상·실패·복구 의미를 설명하고 직접 판정한다.
Repository 확인 위치	cpf-reference transaction EDU 및 실제 Consumer·Test·Config
주요 입력	Actor·Reason·Before/After
실행 순서	입력 Fixture 와 기준 상태 확인 → 정상 실행과 원장 변화 확인 → 로그·Trace·Audit 상호 대조 → 해당 EDU 의 장애 조건 재현 → 상태 Owner 기준 복구 → 동일 입력으로 재검증

항목	수행 내용
정상 판정	업무 Row·History·Audit Commit 시점 일치
장애 재현	Audit 만 Commit 또는 업무만 Commit
복구 판정	Transaction Rollback 후 재시도 가능 상태 확인
운영 확인	/auditLogs
고객 업무 전환	예제 ID·상태·Permission·SLA 만 고객 값으로 바꾸고 Idempotency·Version·Audit·복구 계약은 유지

EDU-ONL-10 — Outbox 원자 저장

항목	수행 내용
학습 결과	Outbox 원자 저장의 선택 기준과 정상·실패·복구 의미를 설명하고 직접 판정한다.
Repository 확인 위치	cpf-reference messaging EDU 및 실제 Consumer·Test·Config
주요 입력	Event Type·Aggregate ID·Payload
실행 순서	입력 Fixture 와 기준 상태 확인 → 정상 실행과 원장 변화 확인 → 로그·Trace·Audit 상호 대조 → 해당 EDU 의 장애 조건 재현 → 상태 Owner 기준 복구 → 동일 입력으로 재검증
정상 판정	업무 Commit 과 Outbox Row 동시 저장
장애 재현	DB Commit 후 Process 종료
복구 판정	Publisher 가 미발행 Row 를 Claim 해 발행
운영 확인	/reliability
고객 업무 전환	예제 ID·상태·Permission·SLA 만 고객 값으로 바꾸고 Idempotency·Version·Audit·복구 계약은 유지

EDU-ONL-11 — Outbox Publisher Lease

항목	수행 내용
학습 결과	Outbox Publisher Lease 의 선택 기준과 정상·실패·복구 의미를 설명하고 직접 판정한다.
Repository 확인 위치	cpf-reference messaging EDU 및 실제 Consumer·Test·Config
주요 입력	Lease Owner·Fencing Token
실행 순서	입력 Fixture 와 기준 상태 확인 → 정상 실행과 원장 변화 확인 → 로그·Trace·Audit 상호 대조 → 해당 EDU 의 장애 조건 재현 → 상태 Owner 기준 복구 → 동일 입력으로 재검증
정상 판정	다중 Instance 중 한 Publisher 만 발행
장애 재현	Lease 만료 뒤 이전 Publisher 늦은 갱신
복구 판정	Fencing Token 으로 stale writer 차단
운영 확인	/topology
고객 업무 전환	예제 ID·상태·Permission·SLA 만 고객 값으로 바꾸고 Idempotency·Version·Audit·복구 계약은 유지

EDU-ONL-12 — Inbox 중복 소비 차단

항목	수행 내용
학습 결과	Inbox 중복 소비 차단의 선택 기준과 정상·실패·복구 의미를 설명하고 직접 판정한다.
Repository 확인 위치	cpf-reference messaging EDU 및 실제 Consumer·Test·Config
주요 입력	Event ID·Consumer ID
실행 순서	입력 Fixture 와 기준 상태 확인 → 정상 실행과 원장 변화 확인 → 로그·Trace·Audit 상호 대조 → 해당 EDU 의 장애 조건 재현 → 상태 Owner 기준 복구 → 동일 입력으로 재검증
정상 판정	재전달 시 기존 처리 결과 반환
장애 재현	ACK 전 Consumer 종료
복구 판정	Inbox 와 업무 결과를 확인하고 ACK 만 재수행
운영 확인	/reliability
고객 업무 전환	예제 ID·상태·Permission·SLA 만 고객 값으로 바꾸고 Idempotency·Version·Audit·복구 계약은 유지

EDU-ONL-13 — Broker DLQ 와 Replay

항목	수행 내용
학습 결과	Broker DLQ 와 Replay 의 선택 기준과 정상·실패·복구 의미를 설명하고 직접 판정한다.
Repository 확인 위치	cpf-reference-messaging-EDU 및 실제 Consumer·Test·Config
주요 입력	DLQ ID·reason·approval
실행 순서	입력 Fixture 와 기준 상태 확인 → 정상 실행과 원장 변화 확인 → 로그·Trace·Audit 상호 대조 → 해당 EDU 의 장애 조건 재현 → 상태 Owner 기준 복구 → 동일 입력으로 재검증
정상 판정	원인 제거 뒤 새 Replay Operation 완료
장애 재현	원인 미해결 Blind Replay
복구 판정	Replay 중단, Payload 와 Consumer Version 분석
운영 확인	/recoveryCenter
고객 업무 전환	예제 ID·상태·Permission·SLA 만 고객 값으로 바꾸고 Idempotency·Version·Audit·복구 계약은 유지

EDU-ONL-14 — Kafka Consumer Process Kill

항목	수행 내용
학습 결과	Kafka Consumer Process Kill 의 선택 기준과 정상·실패·복구 의미를 설명하고 직접 판정한다.
Repository 확인 위치	cpf-reference-fault-EDU 및 실제 Consumer·Test·Config
주요 입력	Fault Point·Event ID
실행 순서	입력 Fixture 와 기준 상태 확인 → 정상 실행과 원장 변화 확인 → 로그·Trace·Audit 상호 대조 → 해당 EDU 의 장애 조건 재현 → 상태 Owner 기준 복구 → 동일 입력으로 재검증
정상 판정	재기동 후 중복 부작용 없이 완료
장애 재현	Commit 후 ACK 전 Kill
복구 판정	Inbox 기준으로 재전달을 정상화
운영 확인	/incidents
고객 업무 전환	예제 ID·상태·Permission·SLA 만 고객 값으로 바꾸고 Idempotency·Version·Audit·복구 계약은 유지

EDU-ONL-15 — RabbitMQ Provider 교체

항목	수행 내용
학습 결과	RabbitMQ Provider 교체의 선택 기준과 정상·실패·복구 의미를 설명하고 직접 판정한다.
Repository 확인 위치	cpf-starters/messaging-rabbitmq 및 실제 Consumer·Test·Config
주요 입력	Exchange·Routing Key·Queue
실행 순서	입력 Fixture 와 기준 상태 확인 → 정상 실행과 원장 변화 확인 → 로그·Trace·Audit 상호 대조 → 해당 EDU 의 장애 조건 재현 → 상태 Owner 기준 복구 → 동일 입력으로 재검증
정상 판정	업무 Event DTO 와 실패 의미 유지
장애 재현	Provider SDK Type 이 Public API 에 노출
복구 판정	Adapter 경계로 이동하고 Contract Test 재실행
운영 확인	/notifications
고객 업무 전환	예제 ID·상태·Permission·SLA 만 고객 값으로 바꾸고 Idempotency·Version·Audit·복구 계약은 유지

EDU-ONL-16 — JMS Provider 교체

항목	수행 내용
학습 결과	JMS Provider 교체의 선택 기준과 정상·실패·복구 의미를 설명하고 직접 판정한다.
Repository 확인 위치	cpf-starters/messaging-jms 및 실제 Consumer·Test·Config
주요 입력	Destination·Ack Mode
실행 순서	입력 Fixture 와 기준 상태 확인 → 정상 실행과 원장 변화 확인 → 로그·Trace·Audit 상호 대조 → 해당 EDU 의 장애 조건 재현 → 상태 Owner 기준 복구 → 동일 입력으로 재검증
정상 판정	표준 Event·Retry·DLQ 계약 유지

항목	수행 내용
장애 재현	Transaction/Ack 의미 불일치
복구 판정	Provider 별 Adapter Test 와 운영 정책 보정
운영 확인	/reliability
고객 업무 전환	예제 ID·상태·Permission·SLA 만 고객 값으로 바꾸고 Idempotency·Version·Audit·복구 계약은 유지

EDU-ONL-17 — IBM MQ 연계

항목	수행 내용
학습 결과	IBM MQ 연계의 선택 기준과 정상·실패·복구 의미를 설명하고 직접 판정한다.
Repository 확인 위치	cpf-starters/messaging-ibm-mq 및 실제 Consumer·Test·Config
주요 입력	Queue Manager·Channel·TLS Secret
실행 순서	입력 Fixture 와 기준 상태 확인 → 정상 실행과 원장 변화 확인 → 로그·Trace·Audit 상호 대조 → 해당 EDU 의 장애 조건 재현 → 상태 Owner 기준 복구 → 동일 입력으로 재검증
정상 판정	MQMD/Correlation 과 CPF 식별자 연결
장애 재현	Channel Down·Backout Queue
복구 판정	Backout 원인 제거 후 승인 Replay
운영 확인	/messages
고객 업무 전환	예제 ID·상태·Permission·SLA 만 고객 값으로 바꾸고 Idempotency·Version·Audit·복구 계약은 유지

EDU-ONL-18 — HTTP Timeout Budget

항목	수행 내용
학습 결과	HTTP Timeout Budget 의 선택 기준과 정상·실패·복구 의미를 설명하고 직접 판정한다.
Repository 확인 위치	cpf-starters/http-client 및 실제 Consumer·Test·Config
주요 입력	상위 Deadline·Connect/Read Timeout
실행 순서	입력 Fixture 와 기준 상태 확인 → 정상 실행과 원장 변화 확인 → 로그·Trace·Audit 상호 대조 → 해당 EDU 의 장애 조건 재현 → 상태 Owner 기준 복구 → 동일 입력으로 재검증
정상 판정	하위 Timeout 합이 상위 Budget 이내
장애 재현	하위 Timeout 이 상위보다 김
복구 판정	정책 검증에서 차단하고 Budget 재배분
운영 확인	/resiliencePolicies
고객 업무 전환	예제 ID·상태·Permission·SLA 만 고객 값으로 바꾸고 Idempotency·Version·Audit·복구 계약은 유지

EDU-ONL-19 — HTTP 응답 유실

항목	수행 내용
학습 결과	HTTP 응답 유실의 선택 기준과 정상·실패·복구 의미를 설명하고 직접 판정한다.
Repository 확인 위치	cpf-reference integration EDU 및 실제 Consumer·Test·Config
주요 입력	Operation ID·Target Request ID
실행 순서	입력 Fixture 와 기준 상태 확인 → 정상 실행과 원장 변화 확인 → 로그·Trace·Audit 상호 대조 → 해당 EDU 의 장애 조건 재현 → 상태 Owner 기준 복구 → 동일 입력으로 재검증
정상 판정	Attempt 가 UNKNOWN_RESULT 로 보존
장애 재현	Target 접수 후 Client Timeout
복구 판정	Status API/대사 파일로 결과 확정
운영 확인	/incidents
고객 업무 전환	예제 ID·상태·Permission·SLA 만 고객 값으로 바꾸고 Idempotency·Version·Audit·복구 계약은 유지

EDU-ONL-20 — Circuit Breaker 전이

항목	수행 내용
학습 결과	Circuit Breaker 전이의 선택 기준과 정상·실패·복구 의미를 설명하고 직접 판정한다.
Repository 확인 위치	cpf-starters/resilience 및 실제 Consumer·Test·Config
주요 입력	실패율·Open Duration·Half-open Probe
실행 순서	입력 Fixture 와 기준 상태 확인 → 정상 실행과 원장 변화 확인 → 로그·Trace·Audit 상호 대조 → 해당 EDU 의 장애 조건 재현 → 상태 Owner 기준 복구 → 동일 입력으로 재검증
정상 판정	CLOSED→OPEN→HALF_OPEN→CLOSED 관측
장애 재현	Retry Storm 과 Thread 고갈
복구 판정	Retry 를 줄이고 Bulkhead·Timeout 을 함께 조정
운영 확인	/resiliencePolicies
고객 업무 전환	예제 ID·상태·Permission·SLA 만 고객 값으로 바꾸고 Idempotency·Version·Audit·복구 계약은 유지

EDU-ONL-21 — TCP Length Framing

항목	수행 내용
학습 결과	TCP Length Framing 의 선택 기준과 정상·실패·복구 의미를 설명하고 직접 판정한다.
Repository 확인 위치	cpf-starters/integration-tcp 및 실제 Consumer·Test·Config
주요 입력	Length Header·Charset·Max Frame
실행 순서	입력 Fixture 와 기준 상태 확인 → 정상 실행과 원장 변화 확인 → 로그·Trace·Audit 상호 대조 → 해당 EDU 의 장애 조건 재현 → 상태 Owner 기준 복구 → 동일 입력으로 재검증
정상 판정	한 Frame 이 정확히 Decode
장애 재현	길이 불일치·Oversize
복구 판정	Connection 종료·원문 격리·재전송 판단
운영 확인	/messages
고객 업무 전환	예제 ID·상태·Permission·SLA 만 고객 값으로 바꾸고 Idempotency·Version·Audit·복구 계약은 유지

EDU-ONL-22 — TCP CRLF Framing

항목	수행 내용
학습 결과	TCP CRLF Framing 의 선택 기준과 정상·실패·복구 의미를 설명하고 직접 판정한다.
Repository 확인 위치	cpf-starters/integration-tcp 및 실제 Consumer·Test·Config
주요 입력	Delimiter·Escape 규칙
실행 순서	입력 Fixture 와 기준 상태 확인 → 정상 실행과 원장 변화 확인 → 로그·Trace·Audit 상호 대조 → 해당 EDU 의 장애 조건 재현 → 상태 Owner 기준 복구 → 동일 입력으로 재검증
정상 판정	경계가 정확한 Message 생성
장애 재현	본문 CRLF 오해석
복구 판정	Escape/Length 방식으로 계약 변경
운영 확인	/messages
고객 업무 전환	예제 ID·상태·Permission·SLA 만 고객 값으로 바꾸고 Idempotency·Version·Audit·복구 계약은 유지

EDU-ONL-23 — TCP STX/ETX Framing

항목	수행 내용
학습 결과	TCP STX/ETX Framing 의 선택 기준과 정상·실패·복구 의미를 설명하고 직접 판정한다.
Repository 확인 위치	cpf-starters/integration-tcp 및 실제 Consumer·Test·Config
주요 입력	STX·ETX·DLE 규칙
실행 순서	입력 Fixture 와 기준 상태 확인 → 정상 실행과 원장 변화 확인 → 로그·Trace·Audit 상호 대조 → 해당 EDU 의 장애 조건 재현 → 상태 Owner 기준 복구 → 동일 입력으로 재검증
정상 판정	Control Byte 포함 Payload 복원
장애 재현	DLE 누락·Frame 중첩

항목	수행 내용
복구 판정	Connection Reset 과 원문 분석
운영 확인	/messages
고객 업무 전환	예제 ID · 상태 · Permission · SLA 만 고객 값으로 바꾸고 Idempotency · Version · Audit · 복구 계약은 유지

EDU-ONL-24 — Fixed-length 전문

항목	수행 내용
학습 결과	Fixed-length 전문의 선택 기준과 정상 · 실패 · 복구 의미를 설명하고 직접 판정한다.
Repository 확인 위치	cpf-starters/integration-fixedlength 및 실제 Consumer · Test · Config
주요 입력	Layout Version · Charset · Padding
실행 순서	입력 Fixture 와 기준 상태 확인 → 정상 실행과 원장 변화 확인 → 로그 · Trace · Audit 상호 대조 → 해당 EDU 의 장애 조건 재현 → 상태 Owner 기준 복구 → 동일 입력으로 재검증
정상 판정	Golden Bytes 와 Object 상호 변환
장애 재현	Byte 길이 · 한글 Charset 오류
복구 판정	원문 보존 후 Layout Version 으로 재파싱
운영 확인	/messages
고객 업무 전환	예제 ID · 상태 · Permission · SLA 만 고객 값으로 바꾸고 Idempotency · Version · Audit · 복구 계약은 유지

EDU-ONL-25 — ISO8583 전문

항목	수행 내용
학습 결과	ISO8583 전문의 선택 기준과 정상 · 실패 · 복구 의미를 설명하고 직접 판정한다.
Repository 확인 위치	cpf-starters/integration-iso8583 및 실제 Consumer · Test · Config
주요 입력	MTI · Bitmap · Field Spec
실행 순서	입력 Fixture 와 기준 상태 확인 → 정상 실행과 원장 변화 확인 → 로그 · Trace · Audit 상호 대조 → 해당 EDU 의 장애 조건 재현 → 상태 Owner 기준 복구 → 동일 입력으로 재검증
정상 판정	Golden Message · MAC · Correlation 일치
장애 재현	Bitmap · MAC · Field Length 오류
복구 판정	NACK · 격리 후 Spec Version 재확인
운영 확인	/messages
고객 업무 전환	예제 ID · 상태 · Permission · SLA 만 고객 값으로 바꾸고 Idempotency · Version · Audit · 복구 계약은 유지

EDU-ONL-26 — Attachment Upload

항목	수행 내용
학습 결과	Attachment Upload 의 선택 기준과 정상 · 실패 · 복구 의미를 설명하고 직접 판정한다.
Repository 확인 위치	cpf-starters/file-attachment 및 실제 Consumer · Test · Config
주요 입력	Size · MIME · Checksum · Owner
실행 순서	입력 Fixture 와 기준 상태 확인 → 정상 실행과 원장 변화 확인 → 로그 · Trace · Audit 상호 대조 → 해당 EDU 의 장애 조건 재현 → 상태 Owner 기준 복구 → 동일 입력으로 재검증
정상 판정	Metadata · Object · Audit 연결
장애 재현	확장자 위장 · Checksum 불일치
복구 판정	격리하고 Object 삭제 또는 재검사
운영 확인	/downloads
고객 업무 전환	예제 ID · 상태 · Permission · SLA 만 고객 값으로 바꾸고 Idempotency · Version · Audit · 복구 계약은 유지

EDU-ONL-27 — Archive 안전 해제

항목	수행 내용
학습 결과	Archive 안전 해제의 선택 기준과 정상·실패·복구 의미를 설명하고 직접 판정한다.
Repository 확인 위치	cpf-starters/file-archive 및 실제 Consumer·Test·Config
주요 입력	Entry Count·Total Size·Path
실행 순서	입력 Fixture와 기준 상태 확인 → 정상 실행과 원장 변화 확인 → 로그·Trace·Audit 상호 대조 → 해당 EDU의 장애 조건 재현 → 상태 Owner 기준 복구 → 동일 입력으로 재검증
정상 판정	허용 경로 아래에 제한된 Entry만 생성
장애 재현	Zip Slip·압축폭탄
복구 판정	전체 해제 중단·임시폴더 삭제·Audit
운영 확인	/file-jobs
고객 업무 전환	예제 ID·상태·Permission·SLA만 고객 값으로 바꾸고 Idempotency·Version·Audit·복구 계약은 유지

EDU-ONL-28 — Tabular Preview·Apply

항목	수행 내용
학습 결과	Tabular Preview·Apply의 선택 기준과 정상·실패·복구 의미를 설명하고 직접 판정한다.
Repository 확인 위치	cpf-starters/file-tabular-poi 및 실제 Consumer·Test·Config
주요 입력	Schema·Header·Row Limit
실행 순서	입력 Fixture와 기준 상태 확인 → 정상 실행과 원장 변화 확인 → 로그·Trace·Audit 상호 대조 → 해당 EDU의 장애 조건 재현 → 상태 Owner 기준 복구 → 동일 입력으로 재검증
정상 판정	Preview 건수·오류 Row·Hash 고정
장애 재현	Apply 중 일부 Row 실패
복구 판정	성공·실패 Row 원장 분리 후 실패만 Reprocess
운영 확인	/file-jobs
고객 업무 전환	예제 ID·상태·Permission·SLA만 고객 값으로 바꾸고 Idempotency·Version·Audit·복구 계약은 유지

EDU-ONL-29 — SFTP 수신

항목	수행 내용
학습 결과	SFTP 수신의 선택 기준과 정상·실패·복구 의미를 설명하고 직접 판정한다.
Repository 확인 위치	cpf-starters/integration-sftp 및 실제 Consumer·Test·Config
주요 입력	Remote Path·Host Key·Checksum
실행 순서	입력 Fixture와 기준 상태 확인 → 정상 실행과 원장 변화 확인 → 로그·Trace·Audit 상호 대조 → 해당 EDU의 장애 조건 재현 → 상태 Owner 기준 복구 → 동일 입력으로 재검증
정상 판정	Temp Download→검증→Atomic Move
장애 재현	중간 끊김·Host Key 변경
복구 판정	부분 파일 삭제·Host Key 승인 확인 후 재수신
운영 확인	/file-jobs
고객 업무 전환	예제 ID·상태·Permission·SLA만 고객 값으로 바꾸고 Idempotency·Version·Audit·복구 계약은 유지

EDU-ONL-30 — Email Notification

항목	수행 내용
학습 결과	Email Notification의 선택 기준과 정상·실패·복구 의미를 설명하고 직접 판정한다.
Repository 확인 위치	cpf-starters/notification-email 및 실제 Consumer·Test·Config
주요 입력	Template·Recipient·Correlation
실행 순서	입력 Fixture와 기준 상태 확인 → 정상 실행과 원장 변화 확인 → 로그·Trace·Audit 상호 대조 → 해당 EDU의 장애 조건 재현 → 상태 Owner 기준 복구 → 동일 입력으로 재검증
정상 판정	Delivery·Receipt·Audit 연결
장애 재현	SMTP Timeout·Bounce

항목	수행 내용
복구 판정	Attempt 상태에 따라 Retry 또는 DLQ
운영 확인	/notifications
고객 업무 전환	예제 ID·상태·Permission·SLA 만 고객 값으로 바꾸고 Idempotency·Version·Audit·복구 계약은 유지

EDU-ONL-31 — SMS SPI

항목	수행 내용
학습 결과	SMS SPI 의 선택 기준과 정상·실패·복구 의미를 설명하고 직접 판정한다.
Repository 확인 위치	cpf-notification-sms-spi 및 실제 Consumer·Test·Config
주요 입력	Template·Phone Token·Provider
실행 순서	입력 Fixture 와 기준 상태 확인 → 정상 실행과 원장 변화 확인 → 로그·Trace·Audit 상호 대조 → 해당 EDU 의 장애 조건 재현 → 상태 Owner 기준 복구 → 동일 입력으로 재검증
정상 판정	원문 전화번호 비노출, Receipt 연결
장애 재현	Provider Timeout·중복 발송
복구 판정	Idempotency 와 Provider Message ID 로 대사
운영 확인	/notifications
고객 업무 전환	예제 ID·상태·Permission·SLA 만 고객 값으로 바꾸고 Idempotency·Version·Audit·복구 계약은 유지

EDU-ONL-32 — Caffeine Local Cache

항목	수행 내용
학습 결과	Caffeine Local Cache 의 선택 기준과 정상·실패·복구 의미를 설명하고 직접 판정한다.
Repository 확인 위치	cpf-starters/cache-caffeine 및 실제 Consumer·Test·Config
주요 입력	Key·TTL·Maximum Size
실행 순서	입력 Fixture 와 기준 상태 확인 → 정상 실행과 원장 변화 확인 → 로그·Trace·Audit 상호 대조 → 해당 EDU 의 장애 조건 재현 → 상태 Owner 기준 복구 → 동일 입력으로 재검증
정상 판정	Owner 조회 감소와 TTL 만료 확인
장애 재현	Stale Entry·Memory Pressure
복구 판정	Namespace Evict 후 Owner 에서 재적재
운영 확인	/cache
고객 업무 전환	예제 ID·상태·Permission·SLA 만 고객 값으로 바꾸고 Idempotency·Version·Audit·복구 계약은 유지

EDU-ONL-33 — Valkey Distributed Cache

항목	수행 내용
학습 결과	Valkey Distributed Cache 의 선택 기준과 정상·실패·복구 의미를 설명하고 직접 판정한다.
Repository 확인 위치	cpf-starters/cache-valkey 및 실제 Consumer·Test·Config
주요 입력	TLS·Key Prefix·Payload Limit
실행 순서	입력 Fixture 와 기준 상태 확인 → 정상 실행과 원장 변화 확인 → 로그·Trace·Audit 상호 대조 → 해당 EDU 의 장애 조건 재현 → 상태 Owner 기준 복구 → 동일 입력으로 재검증
정상 판정	다중 Instance 가 같은 Cache Version 관측
장애 재현	Valkey Down·Partial Key
복구 판정	Fail-open/closed 정책 적용 후 Ledger Reconcile
운영 확인	/cache
고객 업무 전환	예제 ID·상태·Permission·SLA 만 고객 값으로 바꾸고 Idempotency·Version·Audit·복구 계약은 유지

EDU-ONL-34 — Durable Cache Invalidation

항목	수행 내용
학습 결과	Durable Cache Invalidation 의 선택 기준과 정상·실패·복구 의미를 설명하고 직접 판정한다

항목	수행 내용
	다.
Repository 확인 위치	cpf-common cache invalidation 및 실제 Consumer·Test·Config
주요 입력	Event ID·Checkpoint·Consumer ID
실행 순서	입력 Fixture 와 기준 상태 확인 → 정상 실행과 원장 변화 확인 → 로그·Trace·Audit 상호 대조 → 해당 EDU 의 장애 조건 재현 → 상태 Owner 기준 복구 → 동일 입력으로 재검증
정상 판정	Signal 유실 뒤에도 Checkpoint 이후 복구
장애 재현	Pub/Sub Signal 유실
복구 판정	Durable Ledger 를 읽어 누락 Event 재적용
운영 확인	/cache
고객 업무 전환	예제 ID·상태·Permission·SLA 만 고객 값으로 바꾸고 Idempotency·Version·Audit·복구 계약은 유지

EDU-ONL-35 — Feature Flag Override

항목	수행 내용
학습 결과	Feature Flag Override 의 선택 기준과 정상·실패·복구 의미를 설명하고 직접 판정한다.
Repository 확인 위치	cpf-starters/feature-flag 및 실제 Consumer·Test·Config
주요 입력	Typed Value·Rule·Expiry·Approval
실행 순서	입력 Fixture 와 기준 상태 확인 → 정상 실행과 원장 변화 확인 → 로그·Trace·Audit 상호 대조 → 해당 EDU 의 장애 조건 재현 → 상태 Owner 기준 복구 → 동일 입력으로 재검증
정상 판정	평가 Evidence 와 Runtime 결과 일치
장애 재현	만료 누락·잘못된 Target
복구 판정	Override 폐기 또는 Kill Switch
운영 확인	/featureFlags
고객 업무 전환	예제 ID·상태·Permission·SLA 만 고객 값으로 바꾸고 Idempotency·Version·Audit·복구 계약은 유지

EDU-ONL-36 — Secret Rotation

항목	수행 내용
학습 결과	Secret Rotation 의 선택 기준과 정상·실패·복구 의미를 설명하고 직접 판정한다.
Repository 확인 위치	cpf-starters/secret 및 실제 Consumer·Test·Config
주요 입력	Secret Reference·Version·Grace
실행 순서	입력 Fixture 와 기준 상태 확인 → 정상 실행과 원장 변화 확인 → 로그·Trace·Audit 상호 대조 → 해당 EDU 의 장애 조건 재현 → 상태 Owner 기준 복구 → 동일 입력으로 재검증
정상 판정	새 Version 사용과 원문 비노출
장애 재현	일부 Consumer 구 Version
복구 판정	Grace 내 재적용, 실패 Consumer 격리
운영 확인	/secrets
고객 업무 전환	예제 ID·상태·Permission·SLA 만 고객 값으로 바꾸고 Idempotency·Version·Audit·복구 계약은 유지

EDU-ONL-37 — Service Identity

항목	수행 내용
학습 결과	Service Identity 의 선택 기준과 정상·실패·복구 의미를 설명하고 직접 판정한다.
Repository 확인 위치	cpf-starters/security-service-identity 및 실제 Consumer·Test·Config
주요 입력	Audience·Scope·Credential Reference
실행 순서	입력 Fixture 와 기준 상태 확인 → 정상 실행과 원장 변화 확인 → 로그·Trace·Audit 상호 대조 → 해당 EDU 의 장애 조건 재현 → 상태 Owner 기준 복구 → 동일 입력으로 재검증
정상 판정	Service 간 인증·권한 Audit
장애 재현	Wrong Audience·Expired Credential
복구 판정	Credential 교체와 실패 Attempt 확인

항목	수행 내용
운영 확인	/security
고객 업무 전환	예제 ID·상태·Permission·SLA 만 고객 값으로 바꾸고 Idempotency·Version·Audit·복구 계약은 유지

EDU-ONL-38 — Browser Session-CSRF

항목	수행 내용
학습 결과	Browser Session·CSRF의 선택 기준과 정상·실패·복구 의미를 설명하고 직접 판정한다.
Repository 확인 위치	cpf-starters/profile-browser-bff 및 실제 Consumer·Test·Config
주요 입력	Secure Cookie·CSRF Token·Session ID
실행 순서	입력 Fixture와 기준 상태 확인 → 정상 실행과 원장 변화 확인 → 로그·Trace·Audit 상호 대조 → 해당 EDU의 장애 조건 재현 → 상태 Owner 기준 복구 → 동일 입력으로 재검증
정상 판정	JS가 Token 원문에 접근하지 않음
장애 재현	CSRF·Session Fixation·Role Revoke
복구 판정	Session 폐기·재로그인·권한 재평가
운영 확인	/operators
고객 업무 전환	예제 ID·상태·Permission·SLA 만 고객 값으로 바꾸고 Idempotency·Version·Audit·복구 계약은 유지

EDU-ONL-39 — Data Scope 와 Masking

항목	수행 내용
학습 결과	Data Scope와 Masking의 선택 기준과 정상·실패·복구 의미를 설명하고 직접 판정한다.
Repository 확인 위치	cpf-starters/security 및 실제 Consumer·Test·Config
주요 입력	Actor·Organization·Field Policy
실행 순서	입력 Fixture와 기준 상태 확인 → 정상 실행과 원장 변화 확인 → 로그·Trace·Audit 상호 대조 → 해당 EDU의 장애 조건 재현 → 상태 Owner 기준 복구 → 동일 입력으로 재검증
정상 판정	조회·Export에서 다른 Masking 정책 적용
장애 재현	권한 우회·Raw Export
복구 판정	403·Masked 결과·Download Audit 확인
운영 확인	/permissions
고객 업무 전환	예제 ID·상태·Permission·SLA 만 고객 값으로 바꾸고 Idempotency·Version·Audit·복구 계약은 유지

EDU-ONL-40 — Business Calendar

항목	수행 내용
학습 결과	Business Calendar의 선택 기준과 정상·실패·복구 의미를 설명하고 직접 판정한다.
Repository 확인 위치	cpf-common_calendar 및 실제 Consumer·Test·Config
주요 입력	Calendar ID·Date·Day Type·Version
실행 순서	입력 Fixture와 기준 상태 확인 → 정상 실행과 원장 변화 확인 → 로그·Trace·Audit 상호 대조 → 해당 EDU의 장애 조건 재현 → 상태 Owner 기준 복구 → 동일 입력으로 재검증
정상 판정	기준일 계산과 Consumer Refresh 일치
장애 재현	유효기간 겹침·휴일 누락
복구 판정	새 Version 저장 후 Cache Reconcile
운영 확인	/businessCalendar
고객 업무 전환	예제 ID·상태·Permission·SLA 만 고객 값으로 바꾸고 Idempotency·Version·Audit·복구 계약은 유지

EDU-ONL-41 — Time·Deadline

항목	수행 내용
학습 결과	Time·Deadline의 선택 기준과 정상·실패·복구 의미를 설명하고 직접 판정한다.
Repository 확인 위치	cpf-common_time 및 실제 Consumer·Test·Config

항목	수행 내용
주요 입력	UTC Instant · Business Zone · Deadline
실행 순서	입력 Fixture 와 기준 상태 확인 → 정상 실행과 원장 변화 확인 → 로그 · Trace · Audit 상호 대조 → 해당 EDU 의 장애 조건 재현 → 상태 Owner 기준 복구 → 동일 입력으로 재검증
정상 판정	저장 UTC 와 화면 업무시각 일치
장애 재현	Clock Skew · DST 경계
복구 판정	Time Health 확인 후 실행 차단 또는 보정
운영 확인	/integrationClosure
고객 업무 전환	예제 ID · 상태 · Permission · SLA 만 고객 값으로 바꾸고 Idempotency · Version · Audit · 복구 계약은 유지

EDU-ONL-42 — Data Quality Quarantine

항목	수행 내용
학습 결과	Data Quality Quarantine 의 선택 기준과 정상 · 실패 · 복구 의미를 설명하고 직접 판정한다.
Repository 확인 위치	cpf-common data quality 및 실제 Consumer · Test · Config
주요 입력	Rule Set · Record ID · Payload Hash
실행 순서	입력 Fixture 와 기준 상태 확인 → 정상 실행과 원장 변화 확인 → 로그 · Trace · Audit 상호 대조 → 해당 EDU 의 장애 조건 재현 → 상태 Owner 기준 복구 → 동일 입력으로 재검증
정상 판정	위반 Row 격리와 원본 보존
장애 재현	오탐 · Rule Version Drift
복구 판정	Rule Version 확인 후 승인 정정
운영 확인	/integrationClosure
고객 업무 전환	예제 ID · 상태 · Permission · SLA 만 고객 값으로 바꾸고 Idempotency · Version · Audit · 복구 계약은 유지

EDU-ONL-43 — 정정 승인 Snapshot

항목	수행 내용
학습 결과	정정 승인 Snapshot 의 선택 기준과 정상 · 실패 · 복구 의미를 설명하고 직접 판정한다.
Repository 확인 위치	cpf-admin integration closure 및 실제 Consumer · Test · Config
주요 입력	Quarantine ID · expectedVersion · reason
실행 순서	입력 Fixture 와 기준 상태 확인 → 정상 실행과 원장 변화 확인 → 로그 · Trace · Audit 상호 대조 → 해당 EDU 의 장애 조건 재현 → 상태 Owner 기준 복구 → 동일 입력으로 재검증
정상 판정	승인 Snapshot 과 실행 대상 일치
장애 재현	승인 후 Payload 변경 · 만료
복구 판정	실행 차단 후 새 승인 요청
운영 확인	/approvals
고객 업무 전환	예제 ID · 상태 · Permission · SLA 만 고객 값으로 바꾸고 Idempotency · Version · Audit · 복구 계약은 유지

EDU-ONL-44 — OpenAPI Snapshot Refresh

항목	수행 내용
학습 결과	OpenAPI Snapshot Refresh 의 선택 기준과 정상 · 실패 · 복구 의미를 설명하고 직접 판정한다.
Repository 확인 위치	cpf-starters/openapi-webmvc 및 실제 Consumer · Test · Config
주요 입력	Title · Version · Docs Path · Instance
실행 순서	입력 Fixture 와 기준 상태 확인 → 정상 실행과 원장 변화 확인 → 로그 · Trace · Audit 상호 대조 → 해당 EDU 의 장애 조건 재현 → 상태 Owner 기준 복구 → 동일 입력으로 재검증
정상 판정	Schema Hash 와 Controller 일치
장애 재현	중복 Operation ID · Unsafe Path
복구 판정	Schema 수정 후 Refresh 제한을 지켜 재실행
운영 확인	/openApiOperations

항목	수행 내용
고객 업무 전환	예제 ID·상태·Permission·SLA 만 고객 값으로 바꾸고 Idempotency·Version·Audit·복구 계약은 유지

EDU-ONL-45 — Local·Remote 계약 동등성

항목	수행 내용
학습 결과	Local·Remote 계약 동등성의 선택 기준과 정상·실패·복구 의미를 설명하고 직접 판정한다.
Repository 확인 위치	cpf-reference contract fixture 및 실제 Consumer·Test·Config
주요 입력	동일 Request Fixture
실행 순서	입력 Fixture 와 기준 상태 확인 → 정상 실행과 원장 변화 확인 → 로그·Trace·Audit 상호 대조 → 해당 EDU 의 장애 조건 재현 → 상태 Owner 기준 복구 → 동일 입력으로 재검증
정상 판정	응답·Error·Audit 의미 동등
장애 재현	Remote Serialization·Timeout 차이
복구 판정	Adapter 수정 후 같은 Contract Test 재실행
운영 확인	/transactionGroups
고객 업무 전환	예제 ID·상태·Permission·SLA 만 고객 값으로 바꾸고 Idempotency·Version·Audit·복구 계약은 유지

부록 C. 오류·상태·HTTP 판정표

상황	HTTP/상태	재시도 주체	중복 방지	운영자 행동
입력 형식 오류	400	사용자 수정 후 새 요청	해당 없음	반복 발생 시 Client Version 확인
인증 실패	401	Credential 갱신	Session/Token	Security Audit 확인
권한·Data Scope 실패	403	권한 승인 후 새 요청	해당 없음	Role·Scope·Session 재평가
대상 없음	404	ID 확인	해당 없음	삭제·Masking 여부 확인
Version 충돌	409	최신 상태를 읽은 사용자	expectedVersion	자동 재시도 금지
Idempotency Hash 충돌	409	새 Key 발급	key+hash	Client Key 재사용 원인 분석
Rate Limit	429	Client Backoff	Client Request ID	Retry-After 준수
Dispatch 전 연결 실패	FAILED	정책 기반 Retry	idempotencyKey	Attempt 가 전송 전인지 확인
Dispatch 후 응답 유실	UNKNOWN_RESULT	Reconciliation	operationId·targetRequestId	외부 상태조회 전 재전송 금지
Broker 일시 장애	RETRY_SCHEDULED	Publisher/Consumer	Outbox/Inbox	Lag·Retry Age 확인
영구 Payload 오류	DLQ	승인 Replay	eventId·payloadHash	원인 수정 후 새 Replay Operation
일부 Target 적용 실패	PARTIAL	Control Plane	publishId·target	성공 Target 보존, 실패 Target 대상

부록 D. 코드 리뷰에서 반드시 찾을 결함

65. Controller 나 Vue 가 Owner DB 를 직접 접근하는가?
66. Public DTO 에 Kafka·Redis·HTTP Client 구현 Type 이 노출됐는가?
67. 상태 변경 API 가 expectedVersion, reason, idempotencyKey 중 필요한 값을 누락했는가?
68. 업무 Row 와 Audit·Outbox 가 서로 다른 Transaction 인가?
69. 응답을 받지 못한 외부 호출을 곧바로 실패로 기록하는가?
70. Retry 가 HTTP Method 와 Idempotency 보장 여부를 무시하는가?
71. Consumer 가 Inbox 기록보다 ACK 를 먼저 수행하는가?
72. Cache 값을 정보으로 사용하거나 Durable invalidation 없이 Pub/Sub 만 사용하는가?
73. Masking 이 Frontend 에만 있고 Backend·Export 에서 재검증되지 않는가?
74. Unit Test 만 있고 Process Kill·Timeout·중복·동시성·부분 실패 Test 가 없는가?

부록 E. PAY 교육 프로젝트를 실제 업무로 바꾸는 전체 절차

이 부록은 앞의 Source 조각을 파일 단위로 묶어 읽는 순서를 설명한다. 예제 Package 는 고객 프로젝트에서 생성하는 영역이며, cpf-core, cpf-common, cpf-starters 의 내부 Package 는 수정하지 않는다. 실제 CPF 공개 계약 이름은 적용 시점의 JavaDoc 과 cpf-member Consumer 를 기준으로 선택한다.

E.1 학습 결과와 완료 판정

항목	판정 기준
업무 결과	결제수단을 등록하고 일시정지·재개하며 외부기관 등록 결과를 대사한다.
정보	PAY 업무 DB 의 PAY_METHOD, PAY_METHOD_HISTORY, PAY_EXTERNAL_ATTEMPT 가 최종 상태를 소유한다.
중복	같은 Idempotency Key 와 Request Hash 는 기존 결과를 반환하고 부작용은 한 번만 발생한다.
동시성	오래된 Expected Version 은 409 로 차단하고 현재 Version 을 제공한다.
메시지	업무 Commit 과 Outbox 저장에 같은 Transaction 이고 Consumer 는 Inbox 로 중복을 차단한다.
외부기관	Dispatch 이후 Read Timeout 은 UNKNOWN_RESULT 로 남기고 기관 조회로 확정한다.
운영	Transaction ID, Operation ID, Business ID 를 ADM Trace·Incident·Audit 에서 찾을 수 있다.
시험	정상·중복·Payload 충돌·동시수정·Broker 장애·Process Kill·응답 유실을 재현한다.

E.2 고객 프로젝트 Directory

```

pay-domain/
├── build.gradle
├── src/main/java/com/customer/pay/
│   ├── api/
│   │   ├── PayMethodQueryController.java
│   │   ├── PayMethodCommandController.java
│   │   ├── PayErrorHandler.java
│   │   └── dto/
│   ├── application/
│   │   ├── PayMethodQueryService.java
│   │   ├── PayMethodCommandService.java
│   │   ├── PayReconciliationService.java
│   │   └── port/
│   ├── domain/
│   │   ├── PayMethod.java
│   │   ├── PayMethodStatus.java
│   │   ├── PayMethodPolicy.java
│   │   └── event/
│   ├── persistence/
│   │   ├── PayMethodMapper.java
│   │   ├── PayOutboxMapper.java
│   │   ├── PayInboxMapper.java
│   │   └── PayExternalAttemptMapper.java
│   └── src/main/resources/
│       ├── application.yml
│       └── db/migration/
└── src/test/java/com/customer/pay/
    ├── PayMethodCommandTest.java
    ├── PayMethodConcurrencyTest.java
    ├── PayMessagingRecoveryTest.java
    └── PayInstitutionResponseLossTest.java

```

E.3 Gradle 조립 예시

```

plugins {
    id 'java'
    id 'org.springframework.boot'
}

dependencies {
    implementation platform(project(':cpf-tools:build:platform-bom:public-bom'))
    implementation project(':cpf-starters:profiles:secure-api')
    implementation project(':cpf-starters:data-persistence-mybatis')
    implementation project(':cpf-starters:messaging-reliability-jdbc')
    implementation project(':cpf-starters:messaging-kafka')
    implementation project(':cpf-starters:observability')
    implementation project(':cpf-starters:http-client')

    testImplementation project(':cpf-reference')
    testImplementation 'org.springframework.boot:spring-boot-starter-test'
}

tasks.named('test') { useJUnitPlatform() }

```

settings.gradle 의 실제 Project 이름과 Catalog 는 기준 Commit 에서 확인한다. 존재하지 않는 별칭을 문서만 보고 추가하지 않는다. dependencyInsight 로 선택하지 않은 Broker·DB·Session Provider 가 들어오지 않았는지 확인한다.

E.4 Domain 상태 모델

```

package com.customer.pay.domain;

public enum PayMethodStatus {
    PENDING_REGISTRATION,
    ACTIVE,
    SUSPENDED,
    UNKNOWN_RESULT,
}

```

```

    REGISTRATION_FAILED,
    CLOSED
}

public final class PayMethod {
    private final String methodId;
    private final String customerId;
    private final String providerCode;
    private String maskedAccount;
    private PayMethodStatus status;
    private long version;

    public void suspend(long expectedVersion) {
        requireVersion(expectedVersion);
        if (status != PayMethodStatus.ACTIVE) {
            throw new IllegalStateException("ACTIVE 상태에서만 일시정지할 수 있습니다.");
        }
        status = PayMethodStatus.SUSPENDED;
        version++;
    }

    public void resume(long expectedVersion) {
        requireVersion(expectedVersion);
        if (status != PayMethodStatus.SUSPENDED) {
            throw new IllegalStateException("SUSPENDED 상태에서만 재개할 수 있습니다.");
        }
        status = PayMethodStatus.ACTIVE;
        version++;
    }

    public void markUnknown(long expectedVersion) {
        requireVersion(expectedVersion);
        status = PayMethodStatus.UNKNOWN_RESULT;
        version++;
    }

    private void requireVersion(long expectedVersion) {
        if (version != expectedVersion) {
            throw new PayVersionConflict(expectedVersion, version);
        }
    }
}

```

상태전이는 Controller 가 아니라 Domain 에서 검증한다. UNKNOWN_RESULT 에서 ACTIVE 또는 REGISTRATION_FAILED 로 바꾸는 권한은 Reconciliation Service 만 가진다.

E.5 Port 경계

```

public interface PayMethodRepository {
    PayMethod find(String methodId);
    PayMethod lock(String methodId);
    void insert(PayMethod method);
    boolean update(PayMethod method, long expectedVersion);
}

public interface PayIdempotencyStore {
    StoredCommandResult find(String key);
    void reserve(String key, String requestHash, String operationId);
    void complete(String key, String resultHash, String responseJson);
    void fail(String key, String errorCode);
}

public interface PayEventOutbox {
    void append(String eventId, String aggregateId, String eventType, String payloadJson);
}

public interface InstitutionRegistrationPort {
    InstitutionResponse register(String requestId, InstitutionRegistration request);
    InstitutionStatus findStatus(String trackingId, String businessKey);
}

public interface PayAuditPort {
    void append(PayAuditEntry entry);
}

```

Port 이름은 고객 업무에서 정한다. 실제 CPF 기능을 소비할 때는 Public API 또는 SPI Adapter 를 연결하고 Provider SDK Type 을 이 Interface 에 노출하지 않는다.

E.6 Application Service 의 분기

```

@Transactional
public PayMethodResponse register(String idempotencyKey, RegisterPayMethodRequest request) {
    String requestHash = requestHasher.hash(request);
    StoredCommandResult previous = idempotencyStore.find(idempotencyKey);
    if (previous != null) {
        previous.requireSameHash(requestHash);
        return previous.toResponse();
    }

    String operationId = operationIds.next();
    idempotencyStore.reserve(idempotencyKey, requestHash, operationId);
    permission.require("PAY_METHOD_CREATE", request.customerId());

    PayMethod method = PayMethod.pending(ids.next(), request);
    repository.insert(method);
}

```

```

history.appendCreated(method, operationId, request.reason());
audit.append(PayAuditEntry.created(method, operationId, actor.current(), request.reason()));
outbox.append(events.next(), method.methodId(), "PAY_METHOD_REGISTRATION_REQUESTED",
    eventJson.write(method, operationId));

PayMethodResponse response = mapper.toResponse(method, operationId);
idempotencyStore.complete(idempotencyKey, responseHasher.hash(response), json.write(response));
return response;
}

```

외부기관 호출은 이 Transaction 안에서 수행하지 않는다. Outbox Consumer 또는 명시된 Orchestration 단계가 외부 Attempt 를 생성한다.

E.7 Query API 와 검색 방어

```

@GetMapping
@PreAuthorize("hasAuthority('PAY_METHOD_READ')")
public Page<PayMethodSummary> search(
    @RequestParam(required = false) String customerId,
    @RequestParam(required = false) PayMethodStatus status,
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(defaultValue = "20") int size,
    @RequestParam(defaultValue = "updatedAt,desc") String sort) {
    if (size < 1 || size > 200) throw new InvalidPageSize(size);
    SortSpec sortSpec = PaySortWhitelist.parse(sort);
    PaySearchFilter filter = new PaySearchFilter(customerId, status, page, size, sortSpec);
    return queryService.search(filter, actor.current());
}

```

검색 SQL 은 문자열 Sort 를 그대로 붙이지 않는다. 허용 Column 을 Enum 으로 매핑하고 Data Scope 조건을 Query 의 필수 Predicate 로 넣는다.

E.8 DB Table 계약

```

CREATE TABLE PAY_METHOD (
    METHOD_ID          VARCHAR(40) NOT NULL,
    CUSTOMER_ID       VARCHAR(40) NOT NULL,
    PROVIDER_CODE      VARCHAR(30) NOT NULL,
    ACCOUNT_TOKEN_REF VARCHAR(200) NOT NULL,
    MASKED_ACCOUNT     VARCHAR(100) NOT NULL,
    STATUS             VARCHAR(40) NOT NULL,
    VERSION            BIGINT      NOT NULL,
    CREATED_AT         TIMESTAMP   NOT NULL,
    CREATED_BY         VARCHAR(80) NOT NULL,
    UPDATED_AT         TIMESTAMP   NOT NULL,
    UPDATED_BY         VARCHAR(80) NOT NULL,
    CONSTRAINT PK_PAY_METHOD PRIMARY KEY (METHOD_ID)
);

CREATE UNIQUE INDEX UX_PAY_METHOD_PROVIDER
ON PAY_METHOD (CUSTOMER_ID, PROVIDER_CODE, ACCOUNT_TOKEN_REF);

CREATE TABLE PAY_EXTERNAL_ATTEMPT (
    ATTEMPT_ID          VARCHAR(40) NOT NULL,
    OPERATION_ID        VARCHAR(40) NOT NULL,
    METHOD_ID            VARCHAR(40) NOT NULL,
    REQUEST_ID          VARCHAR(80) NOT NULL,
    REQUEST_HASH        VARCHAR(128) NOT NULL,
    TRACKING_ID         VARCHAR(120),
    TARGET_CODE         VARCHAR(40) NOT NULL,
    STATUS              VARCHAR(40) NOT NULL,
    ATTEMPT_NO          INTEGER     NOT NULL,
    VERSION             BIGINT      NOT NULL,
    CREATED_AT          TIMESTAMP   NOT NULL,
    UPDATED_AT          TIMESTAMP   NOT NULL,
    CONSTRAINT PK_PAY_EXTERNAL_ATTEMPT PRIMARY KEY (ATTEMPT_ID)
);

CREATE UNIQUE INDEX UX_PAY_EXTERNAL_REQUEST
ON PAY_EXTERNAL_ATTEMPT (TARGET_CODE, REQUEST_ID);

```

Oracle 에서는 빈 문자열과 VARCHAR2, Sequence·Identity 정책을 Vendor Pack 에서 적용한다. MariaDB·PostgreSQL·Oracle 모두 동일한 Logical Column·Unique 의미를 유지한다.

E.9 MyBatis Update Count 판정

```

<update id="updateStatus">
    UPDATE PAY_METHOD
    SET STATUS = #{status},
        VERSION = VERSION + 1,
        UPDATED_AT = #{updatedAt},
        UPDATED_BY = #{updatedBy}
    WHERE METHOD_ID = #{methodId}
        AND VERSION = #{expectedVersion}
</update>

<select id="findVersion" resultType="long">
    SELECT VERSION
    FROM PAY_METHOD
    WHERE METHOD_ID = #{methodId}
</select>

```

Update Count 가 0 이면 findVersion 으로 존재 여부와 Version 충돌을 분리한다. 재시도로 Update Count 를 숨기지 않는다.

E.10 외부기관 Attempt 상태기계

현재 상태	입력 사건	다음 상태	운영 의미
CREATED	Dispatch 전 연결 실패	FAILED_NOT_SENT	같은 Request ID 로 제한 Retry 가능
CREATED	Request Body 전송 완료	DISPATCHED	부작용 가능성이 생긴 시점
DISPATCHED	2xx 와 Tracking ID 수신	ACCEPTED	기관 상태조회 Key 저장
DISPATCHED	Read Timeout	UNKNOWN_RESULT	재호출 전에 기관 상태조회
ACCEPTED	기관 완료 조회	COMPLETED	PAY 상태 ACTIVE 전이 가능
ACCEPTED	기관 거절 조회	REJECTED	실패 코드와 사유 보존
UNKNOWN_RESULT	기관 완료 확인	COMPLETED	대사 Decision 을 새 이력으로 기록
UNKNOWN_RESULT	기관 미접수 확인	FAILED_NOT_SENT	정책 승인 뒤 새 Attempt 가능
UNKNOWN_RESULT	조회 불가 지속	MANUAL_REVIEW	운영자 대사와 기관 확인 필요

E.11 PowerShell 요청과 예상 응답

```
$base='http://localhost:8080'
$key='edu-pay-register-001'
$body=@{
  customerId='C1001'
  providerCode='BANK-A'
  accountToken='tok_test 001'
  reason='교육용 결제수단 등록'
} | ConvertTo-Json

$r=Invoke-RestMethod -Method Post -Uri "$base/api/pay/methods" `
-headers @{Idempotency-Key=$key} `
-ContentType 'application/json' -Body $body
$r | ConvertTo-Json -Depth 8

# 같은 Key·같은 Body: 같은 methodId와 operationId 계열 결과, 부작용 1 회
$r2=Invoke-RestMethod -Method Post -Uri "$base/api/pay/methods" `
-headers @{Idempotency-Key=$key} `
-ContentType 'application/json' -Body $body
$r2 | ConvertTo-Json -Depth 8
{
  "methodId": "PM-1001",
  "customerId": "C1001",
  "providerCode": "BANK-A",
  "maskedAccount": "****001",
  "status": "PENDING_REGISTRATION",
  "version": 1,
  "operationId": "OP-20260806-000001"
}
```

E.12 DB·Log·Trace·Audit 확인

확인면	검색 Key	정상 결과	이상 징후
업무 DB	METHOD_ID=PM-1001	PAY_METHOD 1 행, VERSION=1	같은 Key 로 중복 Row
Idempotency	Idempotency-Key	Request Hash 와 결과 Snapshot 존재	같은 Key 에 다른 Hash 허용
History	METHOD_ID·OPERATION_ID	CREATED 이력 1 행	업무 Row 와 이력 Commit 시점 불일치
Outbox	Aggregate ID	PENDING 또는 PUBLISHED 1 행	업무 Commit 인데 Outbox 없음
Attempt	OPERATION_ID	외부 호출마다 Attempt No 증가	Timeout 인데 FAILED 로 단정
Log	transactionId·operationId	민감 Token 없이 단계별 구조화 로그	accountToken 원문 노출
Trace	traceld	API→DB→Outbox→Consumer→기관 Segment 연결	Segment 단절·Clock 역전
Audit	actor·operationId	Reason·Before/After·Permission 기록	조치 성공인데 Audit 없음

E.13 자동 Test 전체 목록

Test	Fixture	주입	판정
register_success	신규 고객·Provider	없음	업무·History·Audit·Outbox 동시 Commit
same_key_same_payload	완료된 Idempotency	같은 요청 재호출	같은 결과, 부작용 1 회
same_key_different_payload	예약된 Key	다른 providerCode	409 Key Payload Conflict
optimistic_conflict	VERSION=7	두 Command 동시 실행	한 건 성공, 한 건 409
permission_denied	READ 만 보유	SUSPEND 호출	403, 업무 Row 변화 없음
broker_unavailable	Outbox PENDING	Kafka 연결 실패	업무 Commit 유지, Retry 예정

Test	Fixture	주입	판정
consumer_kill_after_commit	Inbox Transaction	ACK 전 Process 종료	재전달에도 업무 반영 1 회
connect_failure_before_dispatch	Attempt CREATED	DNS/Connect 거부	FAILED_NOT_SENT, 정책 Retry 가능
read_timeout_after_dispatch	Attempt DISPATCHED	응답 지연	UNKNOWN_RESULT, 재호출 금지
reconcile_completed	UNKNOWN_RESULT	기관 완료 응답	Attempt COMPLETED·PAY ACTIVE
reconcile_not_received	UNKNOWN_RESULT	기관 미접수 확인	새 Attempt 전 승인 필요
masking_negative	일반 조회자	상세 조회	Token 원문이 API·Log 에 없음

E.14 Fault Injection 실행 순서

75. 정상 Fixture 와 기대 Row 수를 먼저 기록한다.
76. Toxiproxy 또는 Test Double 에서 장애 지점을 connect, after-dispatch, after-commit, before-ack 로 구분한다.
77. API 응답만 보지 말고 업무 Row·Outbox·Inbox·Attempt·Audit 를 같은 Operation ID 로 조회한다.
78. 장애 후 같은 Key 를 재호출하기 전에 상태가 확정됐는지 확인한다.
79. Reconciliation 은 원본 Attempt 를 수정해서 지우지 않고 Decision 이력을 추가한다.
80. 정상화 뒤 건수·상태·외부 등록 횟수를 다시 계산한다.

E.15 고객 업무 전환표

예제 값	고객이 바꿀 것	유지할 계약
PayMethod	고객의 신청·계좌·주문 Aggregate	Owner 가 최종 상태를 소유
ACTIVE/SUSPENDED	고객 상태와 허용 전이	Expected Version 과 상태전이 검증
BANK-A	실제 외부기관·Channel	Attempt Ledger 와 Timeout 분류
PAY_METHOD_WRITE	고객 Permission Code	Menu·Button·API Permission 일치
10 초 Timeout	SLA 와 기관 계약	Timeout Budget 과 Dispatch 전후 분류
Kafka Topic	실제 Event Type·Partition Key	Outbox·Inbox·Schema Version
PAY_METHOD Table	고객 Column·Index	History·Audit·Unique·Version
ADM Route	업무 Route·Field·Button	Query/Command 분리와 Operation 조회

부록 F. 개발 중 자주 만나는 20 개 판단 사례

F.1 검색 결과가 2 만 건을 넘는다

관점	내용
영역	조회
먼저 확인할 사실	검색 결과가 2 만 건을 넘는다가 발생한 시점의 Transaction ID·Operation ID·Business ID 와 부작용 경계를 찾는다.
설계 판단	최대 Page Size 를 낮추고 Cursor 또는 Keyset Paging 을 검토
금지 판단	무제한 Export 를 일반 조회 API 로 처리
시험	정상 Fixture 와 실패 Fixture 를 분리하고 검색 결과가 2 만 건을 넘는다 조건을 재현한다.
완료 근거	Query Plan·응답 크기·메모리·Download Audit
운영 연계	ADM 에서 검색할 식별자, 허용 조치, 종료 판정을 Query Plan·응답 크기·메모리·Download Audit 와 함께 기록한다.

F.2 두 사용자가 같은 신청을 승인한다

관점	내용
영역	동시성
먼저 확인할 사실	두 사용자가 같은 신청을 승인한다가 발생한 시점의 Transaction ID·Operation ID·Business ID 와 부작용 경계를 찾는다.
설계 판단	Expected Version 과 승인 Snapshot 으로 한 명만 성공
금지 판단	Last-write-wins 또는 Blind Retry
시험	정상 Fixture 와 실패 Fixture 를 분리하고 두 사용자가 같은 신청을 승인한다 조건을 재현한다.
완료 근거	409 응답·현재 Version·Audit 두 건

관점	내용
운영 인계	ADM 에서 검색할 식별자, 허용 조치, 종료 판정을 409 응답·현재 Version·Audit 두건와 함께 기록한다.

F.3 기관이 처리했지만 HTTP 응답이 끊겼다

관점	내용
영역	외부 연계
먼저 확인할 사실	기관이 처리했지만 HTTP 응답이 끊겼다가 발생한 시점의 Transaction ID·Operation ID·Business ID 와 부작용 경계를 찾는다.
설계 판단	Attempt 를 UNKNOWN_RESULT 로 남기고 Tracking ID 조회
금지 판단	같은 요청을 새 Request ID 로 재전송
시험	정상 Fixture 와 실패 Fixture 를 분리하고 기관이 처리했지만 HTTP 응답이 끊겼다 조건을 재현한다.
완료 근거	기관 등록 횟수·Attempt History·대사 Decision
운영 인계	ADM 에서 검색할 식별자, 허용 조치, 종료 판정을 기관 등록 횟수·Attempt History·대사 Decision 와 함께 기록한다.

F.4 Broker 는 10 분간 사용할 수 없다

관점	내용
영역	메시징
먼저 확인할 사실	Broker 는 10 분간 사용할 수 없다가 발생한 시점의 Transaction ID·Operation ID·Business ID 와 부작용 경계를 찾는다.
설계 판단	업무 Commit 과 Outbox 를 보존하고 Backoff Retry
금지 판단	업무 Transaction 을 Broker 복구까지 유지
시험	정상 Fixture 와 실패 Fixture 를 분리하고 Broker 는 10 분간 사용할 수 없다 조건을 재현한다.
완료 근거	Outbox Lag·Retry Count·Circuit 상태
운영 인계	ADM 에서 검색할 식별자, 허용 조치, 종료 판정을 Outbox Lag·Retry Count·Circuit 상태와 함께 기록한다.

F.5 Consumer 가 DB Commit 뒤 종료됐다

관점	내용
영역	메시징
먼저 확인할 사실	Consumer 가 DB Commit 뒤 종료됐다가 발생한 시점의 Transaction ID·Operation ID·Business ID 와 부작용 경계를 찾는다.
설계 판단	Inbox 와 업무 Commit 후 재전달을 중복 차단
금지 판단	ACK 여부만 보고 업무를 다시 수행
시험	정상 Fixture 와 실패 Fixture 를 분리하고 Consumer 가 DB Commit 뒤 종료됐다 조건을 재현한다.
완료 근거	Inbox Unique·업무 Row 수·Broker Offset
운영 인계	ADM 에서 검색할 식별자, 허용 조치, 종료 판정을 Inbox Unique·업무 Row 수·Broker Offset 와 함께 기록한다.

F.6 파일 100 만 행 중 23 행이 오류다

관점	내용
영역	파일
먼저 확인할 사실	파일 100 만 행 중 23 행이 오류다가 발생한 시점의 Transaction ID·Operation ID·Business ID 와 부작용 경계를 찾는다.
설계 판단	Preview·행별 결과 원장·부분 적용 정책
금지 판단	첫 오류에서 전체 상태를 성공으로 종료
시험	정상 Fixture 와 실패 Fixture 를 분리하고 파일 100 만 행 중 23 행이 오류다 조건을 재현한다.

관점	내용
완료 근거	입력=성공+실패+제외, Artifact Hash
운영 인계	ADM 에서 검색할 식별자, 허용 조치, 종료 판정을 입력=성공+실패+제외, Artifact Hash 와 함께 기록한다.

F.7 권한이 회수됐지만 Session 이 남았다

관점	내용
영역	보안
먼저 확인할 사실	권한이 회수됐지만 Session 이 남았다가 발생한 시점의 Transaction ID·Operation ID·Business ID 와 부작용 경계를 찾는다.
설계 판단	Session 재평가·강제 폐기·API 권한 재검증
금지 판단	메뉴 숨김만 적용
시험	정상 Fixture 와 실패 Fixture 를 분리하고 권한이 회수됐지만 Session 이 남았다 조건을 재현한다.
완료 근거	Session Row·403 Test·Audit
운영 인계	ADM 에서 검색할 식별자, 허용 조치, 종료 판정을 Session Row·403 Test·Audit 와 함께 기록한다.

F.8 Secret Key Version 을 교체한다

관점	내용
영역	보안
먼저 확인할 사실	Secret Key Version 을 교체한다가 발생한 시점의 Transaction ID·Operation ID·Business ID 와 부작용 경계를 찾는다.
설계 판단	새 Version 배포→Consumer ACK→Grace→구 Version 폐기
금지 판단	모든 Consumer 확인 전 구 Key 폐기
시험	정상 Fixture 와 실패 Fixture 를 분리하고 Secret Key Version 을 교체한다 조건을 재현한다.
완료 근거	Target 별 Active Version·Decrypt Test
운영 인계	ADM 에서 검색할 식별자, 허용 조치, 종료 판정을 Target 별 Active Version·Decrypt Test 와 함께 기록한다.

F.9 DB Migration 뒤 구 버전 Instance 가 남았다

관점	내용
영역	호환성
먼저 확인할 사실	DB Migration 뒤 구 버전 Instance 가 남았다가 발생한 시점의 Transaction ID·Operation ID·Business ID 와 부작용 경계를 찾는다.
설계 판단	Expand/Contract 단계와 Mixed Version Test
금지 판단	파괴적 DDL 을 먼저 적용
시험	정상 Fixture 와 실패 Fixture 를 분리하고 DB Migration 뒤 구 버전 Instance 가 남았다 조건을 재현한다.
완료 근거	Schema Version·Instance Version·Rollback 가능성
운영 인계	ADM 에서 검색할 식별자, 허용 조치, 종료 판정을 Schema Version·Instance Version·Rollback 가능성 와 함께 기록한다.

F.10 같은 업무를 Local 에서 Remote 로 분리한다

관점	내용
영역	Topology
먼저 확인할 사실	같은 업무를 Local 에서 Remote 로 분리한다가 발생한 시점의 Transaction ID·Operation ID·Business ID 와 부작용 경계를 찾는다.
설계 판단	DTO·Validation·Error·Idempotency Contract Test 재사용
금지 판단	Remote 전환과 함께 업무 규칙 변경
시험	정상 Fixture 와 실패 Fixture 를 분리하고 같은 업무를 Local 에서 Remote 로 분리한

관점	내용
	다 조건을 재현한다.
완료 근거	Local/Remote Fixture 결과·Trace
운영 인계	ADM 에서 검색할 식별자, 허용 조치, 종료 판정을 Local/Remote Fixture 결과·Trace 와 함께 기록한다.

F.11 다중 Instance 에서 Cache 값이 다르다

관점	내용
영역	Cache
먼저 확인할 사실	다중 Instance 에서 Cache 값이 다르다가 발생한 시점의 Transaction ID·Operation ID·Business ID 와 부작용 경계를 찾는다.
설계 판단	Durable invalidation 원장과 Checkpoint 로 Reconcile
금지 판단	Pub/Sub Signal 만 정보으로 사용
시험	정상 Fixture 와 실패 Fixture 를 분리하고 다중 Instance 에서 Cache 값이 다르다 조건을 재현한다.
완료 근거	Version·Checkpoint·재적재 건수
운영 인계	ADM 에서 검색할 식별자, 허용 조치, 종료 판정을 Version·Checkpoint·재적재 건수와 함께 기록한다.

F.12 외부 요청 Body 가 50MB 다

관점	내용
영역	Resource
먼저 확인할 사실	외부 요청 Body 가 50MB 다가 발생한 시점의 Transaction ID·Operation ID·Business ID 와 부작용 경계를 찾는다.
설계 판단	Bounded Streaming·Size Limit·Temp Cleanup
금지 판단	전체 Body 를 Memory 에 적재
시험	정상 Fixture 와 실패 Fixture 를 분리하고 외부 요청 Body 가 50MB 다 조건을 재현한다.
완료 근거	Heap·Disk·Checksum·Cleanup
운영 인계	ADM 에서 검색할 식별자, 허용 조치, 종료 판정을 Heap·Disk·Checksum·Cleanup 와 함께 기록한다.

F.13 고객이 같은 파일을 다시 업로드했다

관점	내용
영역	파일
먼저 확인할 사실	고객이 같은 파일을 다시 업로드했다가 발생한 시점의 Transaction ID·Operation ID·Business ID 와 부작용 경계를 찾는다.
설계 판단	Checksum·Business Key·Idempotency 로 중복 판정
금지 판단	파일명만 비교
시험	정상 Fixture 와 실패 Fixture 를 분리하고 고객이 같은 파일을 다시 업로드했다 조건을 재현한다.
완료 근거	Upload Hash·Job ID·행 결과
운영 인계	ADM 에서 검색할 식별자, 허용 조치, 종료 판정을 Upload Hash·Job ID·행 결과와 함께 기록한다.

F.14 정정 승인 후 원본 Row 가 바뀌었다

관점	내용
영역	승인
먼저 확인할 사실	정정 승인 후 원본 Row 가 바뀌었다가 발생한 시점의 Transaction ID·Operation ID·Business ID 와 부작용 경계를 찾는다.
설계 판단	Approval Snapshot 과 Expected Version 으로 실행 거부
금지 판단	현재 값을 조용히 덮어쓰

관점	내용
시험	정상 Fixture 와 실패 Fixture 를 분리하고 정정 승인 후 원본 Row 가 바뀌었다 조건을 재현한다.
완료 근거	Snapshot Hash · Version Conflict · 새 승인
운영 연계	ADM 에서 검색할 식별자, 허용 조치, 종료 판정을 Snapshot Hash · Version Conflict · 새 승인 와 함께 기록한다.

F.15 OpenAPI Operation ID 가 중복됐다

관점	내용
영역	계약
먼저 확인할 사실	OpenAPI Operation ID 가 중복됐다가 발생한 시점의 Transaction ID · Operation ID · Business ID 와 부작용 경계를 찾는다.
설계 판단	Build Gate 에서 실패시키고 Generated Client 를 갱신
금지 판단	마지막 Controller 가 이전 정의를 덮어씀
시험	정상 Fixture 와 실패 Fixture 를 분리하고 OpenAPI Operation ID 가 중복됐다 조건을 재현한다.
완료 근거	Schema Diff · Operation 목록 · Client Build
운영 연계	ADM 에서 검색할 식별자, 허용 조치, 종료 판정을 Schema Diff · Operation 목록 · Client Build 와 함께 기록한다.

F.16 시간대가 다른 기관과 영업일을 계산한다

관점	내용
영역	시간
먼저 확인할 사실	시간대가 다른 기관과 영업일을 계산하다가 발생한 시점의 Transaction ID · Operation ID · Business ID 와 부작용 경계를 찾는다.
설계 판단	UTC 저장 · 업무 Zone 명시 · Calendar Version
금지 판단	서버 Local Time 을 묵시 사용
시험	정상 Fixture 와 실패 Fixture 를 분리하고 시간대가 다른 기관과 영업일을 계산한다 조건을 재현한다.
완료 근거	입력 Zone · 계산일 · Calendar Version
운영 연계	ADM 에서 검색할 식별자, 허용 조치, 종료 판정을 입력 Zone · 계산일 · Calendar Version 와 함께 기록한다.

F.17 개인정보 Export 가 오래 걸린다

관점	내용
영역	Download
먼저 확인할 사실	개인정보 Export 가 오래 걸린다가 발생한 시점의 Transaction ID · Operation ID · Business ID 와 부작용 경계를 찾는다.
설계 판단	비동기 Job · 만료 Token · 권한 재검증
금지 판단	브라우저 요청을 장시간 유지
시험	정상 Fixture 와 실패 Fixture 를 분리하고 개인정보 Export 가 오래 걸린다 조건을 재현한다.
완료 근거	Job 상태 · Download Audit · 만료
운영 연계	ADM 에서 검색할 식별자, 허용 조치, 종료 판정을 Job 상태 · Download Audit · 만료 와 함께 기록한다.

F.18 Retry 가 전체 Thread 를 소진한다

관점	내용
영역	Resilience
먼저 확인할 사실	Retry 가 전체 Thread 를 소진하다가 발생한 시점의 Transaction ID · Operation ID · Business ID 와 부작용 경계를 찾는다.
설계 판단	Budget · Backoff · Bulkhead · Circuit 을 함께 적용

관점	내용
금지 판단	무제한 Retry
시험	정상 Fixture 와 실패 Fixture 를 분리하고 Retry 가 전체 Thread 를 소진한다 조건을 재현한다.
완료 근거	Queue·Thread·Attempt Count·Circuit State
운영 인계	ADM 에서 검색할 식별자, 허용 조치, 종료 판정을 Queue·Thread·Attempt Count·Circuit State 와 함께 기록한다.

F.19 배포 도중 Error Rate 가 증가한다

관점	내용
영역	배포
먼저 확인할 사실	배포 도중 Error Rate 가 증가하다가 발생한 시점의 Transaction ID·Operation ID·Business ID 와 부작용 경계를 찾는다.
설계 판단	Canary 중단·LKG 복귀·Unknown Operation 대사
금지 판단	모든 Target 배포 뒤 판단
시험	정상 Fixture 와 실패 Fixture 를 분리하고 배포 도중 Error Rate 가 증가한다 조건을 재현한다.
완료 근거	Target Version·SLO Window·Rollback Audit
운영 인계	ADM 에서 검색할 식별자, 허용 조치, 종료 판정을 Target Version·SLO Window·Rollback Audit 와 함께 기록한다.

F.20 운영자가 결과를 수동 확정한다

관점	내용
영역	운영
먼저 확인할 사실	운영자가 결과를 수동 확정한다가 발생한 시점의 Transaction ID·Operation ID·Business ID 와 부작용 경계를 찾는다.
설계 판단	근거 원장·Reason·Approval·Decision 이력 요구
금지 판단	DB 직접 Update
시험	정상 Fixture 와 실패 Fixture 를 분리하고 운영자가 결과를 수동 확정한다 조건을 재현한다.
완료 근거	Operation·Before/After·Approver·Audit
운영 인계	ADM 에서 검색할 식별자, 허용 조치, 종료 판정을 Operation·Before/After·Approver·Audit 와 함께 기록한다.

DB Upgrade 는 Migration 적용 자체로 끝나지 않는다. Upgrade 전 Schema Version·Vendor Pack·Backup 시점을 고정하고, 구 Version Instance 와의 호환 구간을 확인한 뒤 적용한다. 실패하면 적용된 Migration 범위와 데이터 변경 여부를 확인해 Rollback Script 또는 Restore 를 선택하며, 데이터가 변경된 경우 Schema Rollback 만으로 복구 완료를 선언하지 않는다. 외부기관 Read Timeout 뒤에는 새 요청을 먼저 보내지 않고 Attempt ID 로 Status Query(상태 조회)를 수행해 기관 처리 여부를 확인한 후 Retry 또는 Reconcile 을 선택한다.

[↑ 문서 목차로 이동](#)